ImageGuard - Detection of Fake Images in Online Marketplaces Using Deep Learning and Explainable AI

PROJECT PRESENTATION

Guide: Dr. Preetha K G

Team Members:
Aleena P
Alen Biju Nilakkappillil
Anand Basil
Anna Praveen

CONTENTS

- Introduction
- Problem Statement
- Implementation Scope
- Literature Survey
- Architecture Diagram
- Sequence Diagram
- Module Division
- Role of Each Individual
- Hardware and Software Requirements
- Project Schedule
- Results
- Assumptions
- Risk and Challenges
- Conclusion
- Future Plans
- References

INTRODUCTION

- Online marketplaces heavily rely on product images to attract buyers.
- Rise in fake, manipulated, and AI-generated images misleads consumers.
- Fake images cause financial loss, trust issues, and platform credibility damage.
- Manual image verification is time-consuming and not scalable.
- Deep Learning enables automatic detection of image forgery.
- Lack of transparency in DL models creates trust and explainability concerns.
- ImageGuard integrates Deep Learning + Explainable AI (XAI) Provides accurate detection along with visual explanations of decisions.
- Available as a browser extension to promote safe and transparent online shopping.

PROBLEM STATEMENT

- To design and develop a deep learning–based model for detecting fake product images in online marketplaces.
- The system will analyze visual patterns, product description inconsistencies, and image manipulations to differentiate between authentic and fake/AI-generated images.

IMPLEMENTATION SCOPE

1. Model Development: Build and train CNN (EfficientNet) and ViT models to classify images as real or fake.
2. Explainability: Implement Grad-CAM and SHAP to visually explain why an image is flagged as fake.
3. Multimodal Verification: Integrate YOLO and Regex to check consistency between product images and descriptions.
4. Deployment: Deliver real-time detection via an inference API and a lightweight browser extension.

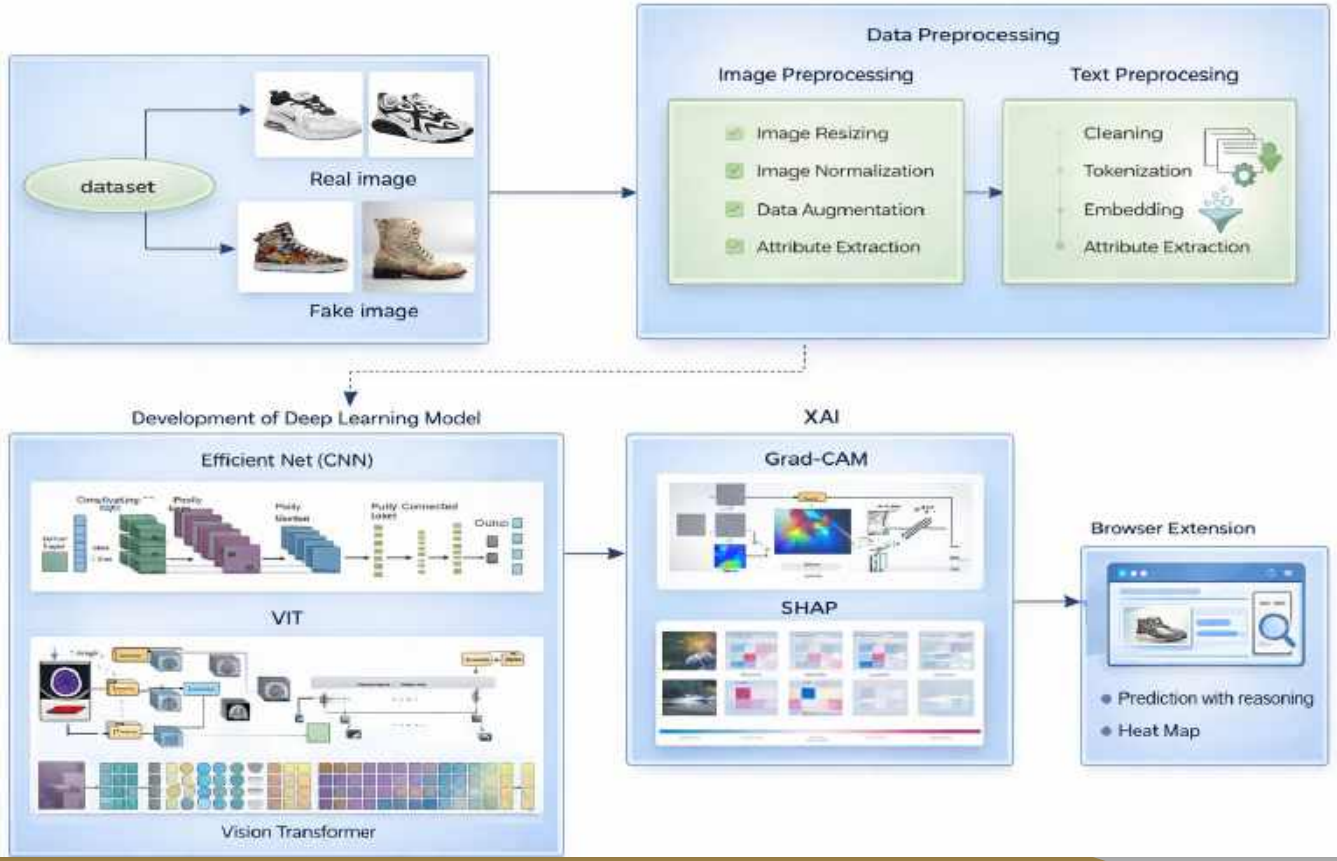
Literature Review

Paper Title	Methodology	Strengths	Limitations
Detecting Fake Images on Social Media using Machine Learning [1]	CNN extracts features using convolutional layers, pooling, and activation functions	Learns visual patterns automatically, high accuracy in image classification	Needs large datasets and GPU support
Deepfake Image Detection using Vision Transformer Models [2]	ViT patches the image, encodes the position of each patch, and applies multi-head attention and MLP layers	Understands full image context accurately, better generalization	Requires more data, high computational cost
Harnessing eXplainable artificial intelligence for feature selection in time series energy forecasting: A comparative analysis of Grad-CAM and SHAP [3]	XAI analyzes data by making model predictions transparent and comprehensible	Provides clear visual explanations for model decisions	Adds extra steps and complexity to the system

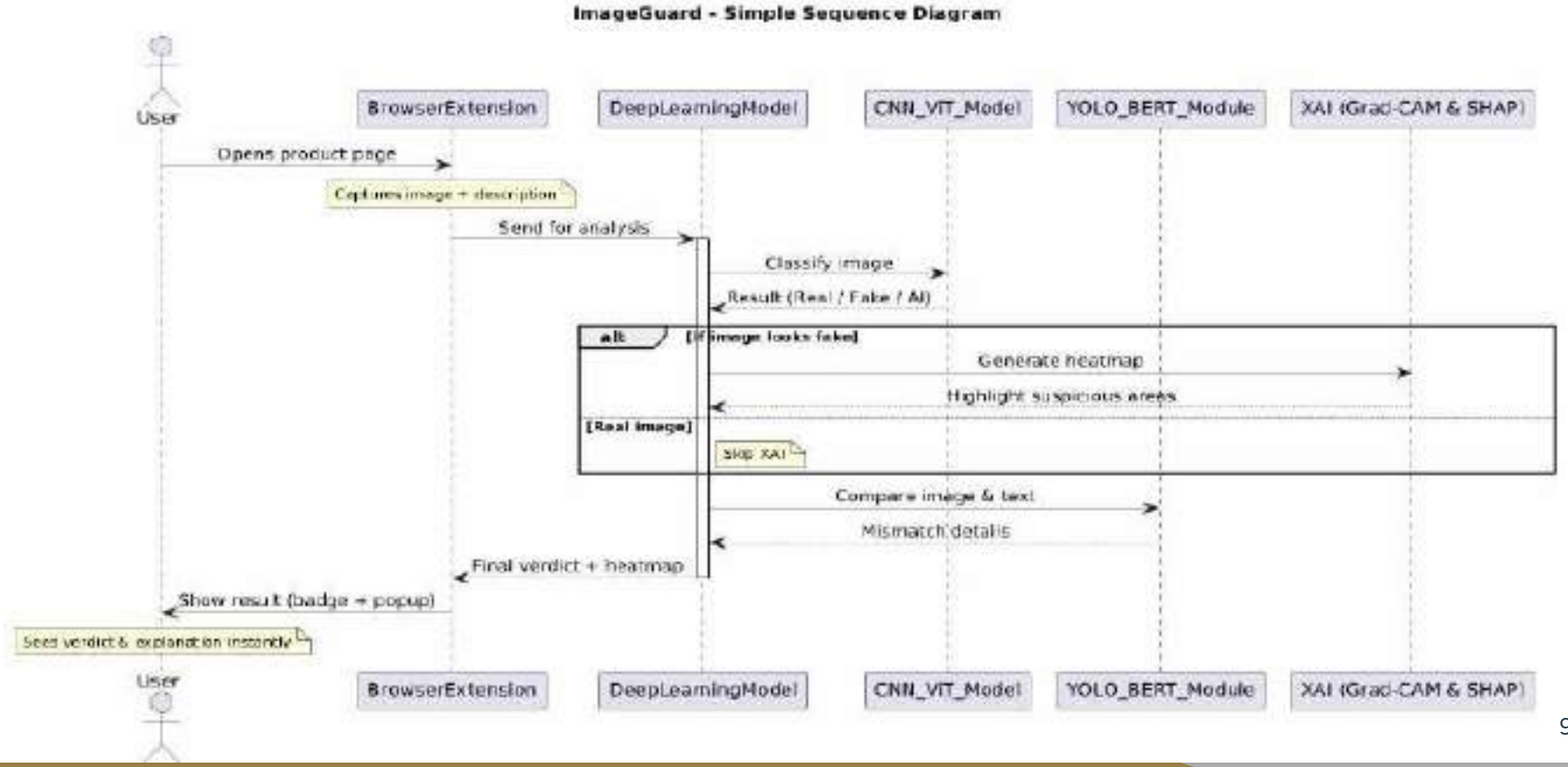
Literature Review

Paper Title	Methodology	Strengths	Limitations
An Introduction to Image Classification and Object Detection using YOLO Detector [4]	YOLO divides images into a grid, predicting bounding boxes and class labels	Fast and accurate object detection in one step	May miss small or overlapping objects
Fake Image Detection Using Deep Learning [5]	EfficientNet uses a compound scaling method to uniformly scale depth, width, and input resolution to achieve better performance with optimal efficiency.	EfficientNet provides high classification accuracy with fewer parameters and lower computational cost compared to traditional deep CNNs.	EfficientNet requires careful hyperparameter tuning and higher computational resources for larger variants like B4–B7

OVERALL ARCHITECTURE DIAGRAM



SEQUENCE DIAGRAM



MODULE DIVISION

1. Dataset Preparation
2. Development of Deep Learning Model (CNN)
3. Development of Deep Learning Model (ViT)
4. Explainable AI (Grad-CAM + SHAP)
5. Product Description Analysis
6. Browser Extension (Frontend Integration)

ROLE OF EACH INDIVIDUAL

Alen Biju: Data collection and preprocessing, Chrome extension development and integration.

Anand Basil: Explainable AI (Grad-CAM, SHAP), Model training, Backend integration.

Aleena P: Vision Transformer (ViT-Base) implementation, Product description checking module.

Anna Praveen: CNN development (EfficientNet-B2), Dataset preparation, Product description verification.

HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- System with minimum Intel i5 / Ryzen 5 processor
- At least 8GB RAM (16GB recommended for training)
- NVIDIA GPU (4GB+ VRAM recommended) for model training
- Minimum 20GB free storage for dataset and models

Software Requirements

- Frontend: HTML, CSS, JavaScript, Chrome Extension
- Backend: Python 3.8+, FastAPI (Python)
- Machine Learning: PyTorch, timm (EfficientNet-B2, ViT-Base), SHAP library, Grad-CAM
- Dataset: Manually prepared

PROJECT SCHEDULE

1. DATASET PREPARATION

- Collection of authentic and fake / AI-generated product images
- Data sourced from online marketplaces and synthetic generation tools (CycleGAN)
- Labeling of images as Real or Fake
- Removal of noisy, duplicate, and low-quality samples
- Dataset split into training, validation, and testing sets

Shoes:- Real: 3757 ; Fake: 5199
Watches:- Real: 6897 ; Fake 5669

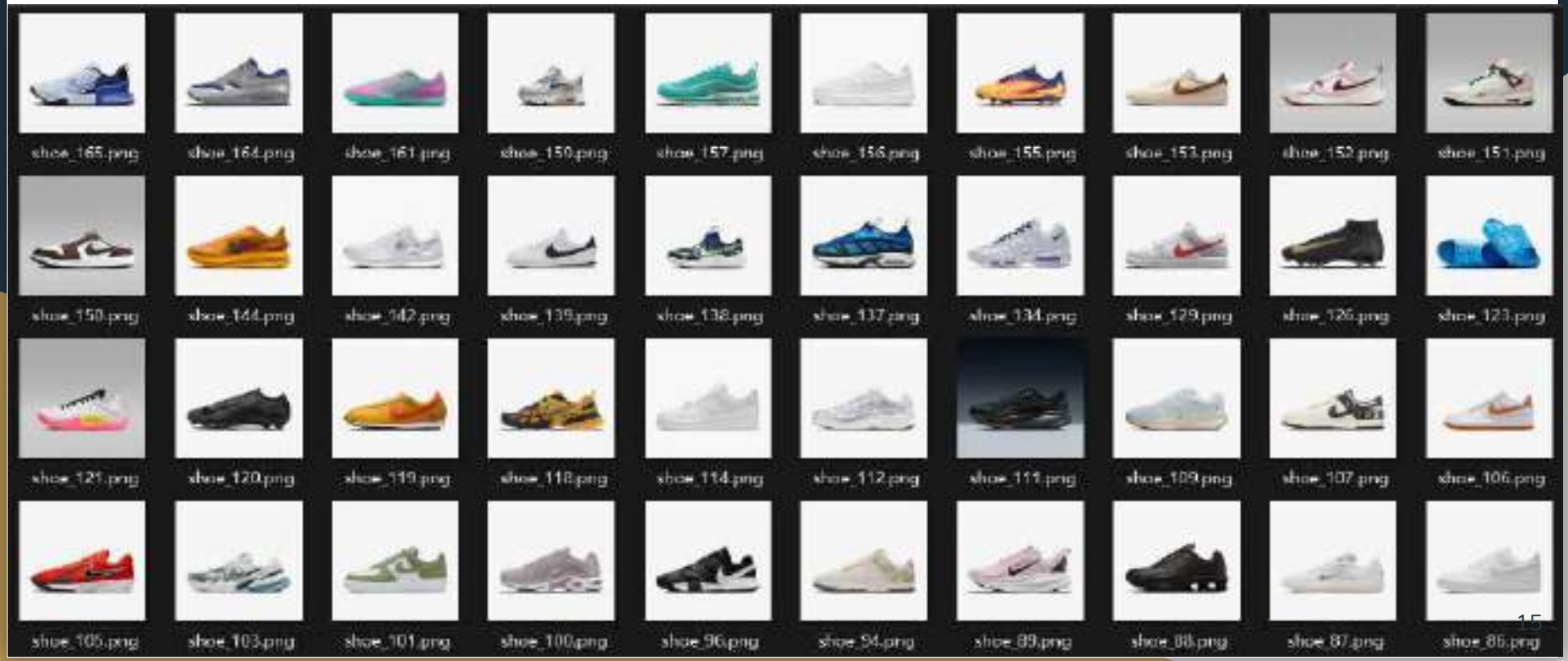
Real Images: 10654 images

Fake Images: 10868 images

```
dataset/
├── train (70%)
│   ├── real  (7223 images)
│   └── fake  (7623 images)
├── val (15%)
│   ├── real  (1714 images)
│   └── fake  (1745 images)
└── test (15%)
    ├── real  (1717 images)
    └── fake  (1500 images)
```

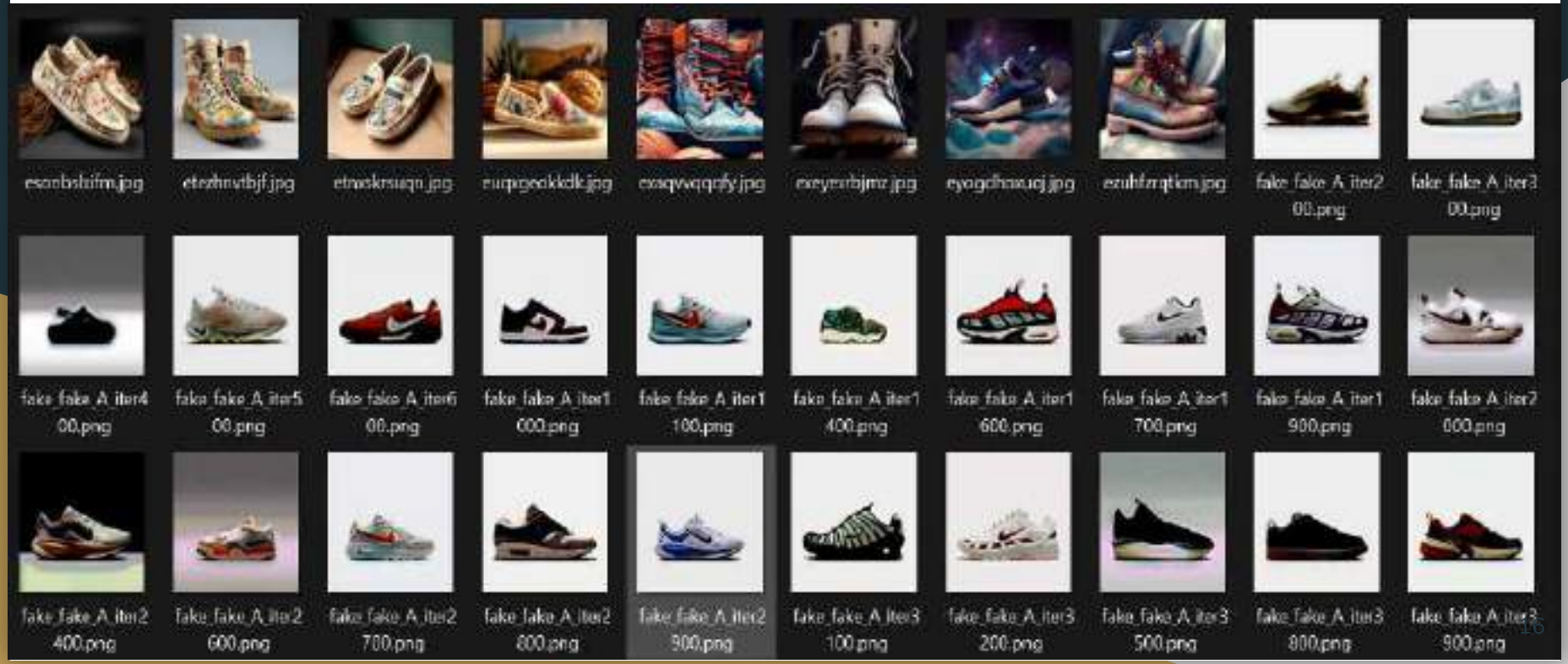
1. DATASET PREPARATION

Dataset Creation - Real



1. DATASET PREPARATION

Dataset Creation - Fake



2. DEVELOPMENT OF DEEP LEARNING MODEL (CNN)

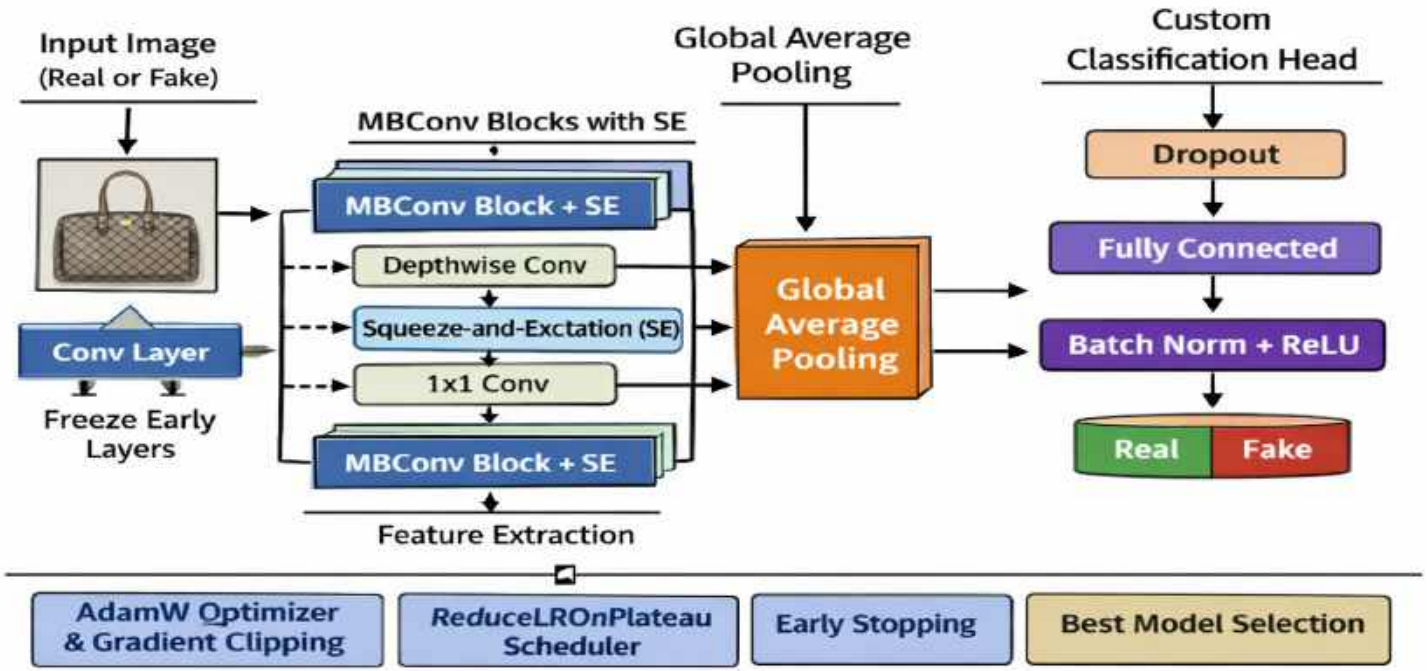
- EfficientNet-B2 is a convolutional neural network designed to achieve high accuracy with computational efficiency.
- It uses convolution operations to extract spatial features from images.
- The model applies compound scaling, which balances network depth, width, and input resolution for optimal performance.
- EfficientNet focuses on learning local patterns such as edges, textures, and fine details, making it effective for visual recognition tasks.
- It enables efficient feature extraction with fewer parameters, allowing detection of subtle texture inconsistencies while maintaining fast inference speed.

2. DEVELOPMENT OF DEEP LEARNING MODEL (CNN)

Working of EfficientNet-B2

- The input image is first resized to a fixed resolution and passed through an initial convolution layer that captures basic visual features such as edges and color gradients.
- The feature maps are then processed through multiple MBConv (Mobile Inverted Bottleneck) blocks, which use depthwise separable convolutions to efficiently learn spatial and channel-wise patterns while reducing computational cost.
- Within these blocks, Squeeze-and-Excitation (SE) modules adaptively recalibrate channel importance, allowing the network to focus on the most informative features.
- After feature extraction, a global average pooling layer converts the spatial feature maps into a compact feature vector representing the entire image.
- Finally, the feature vector is passed through a fully connected classification layer, enabling the model to make predictions based on the learned hierarchical representations.

2. DEVELOPMENT OF DEEP LEARNING MODEL (CNN)



EfficientNet-B2 for Fake Image Detection

3. DEVELOPMENT OF DEEP LEARNING MODEL (ViT)

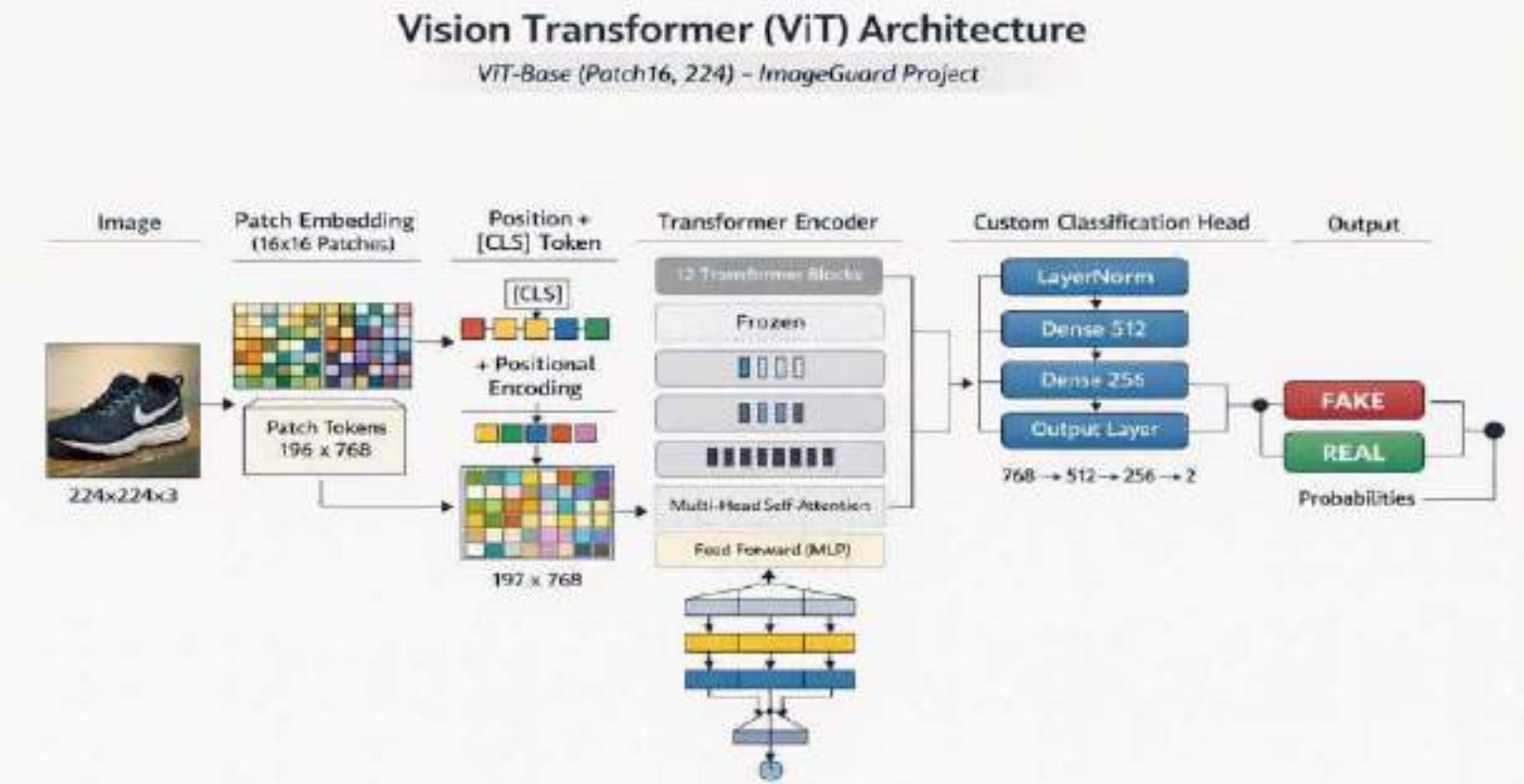
- Vision Transformer (ViT) applies the **Transformer architecture** (originally from NLP) to images.
- Instead of convolutions, it treats image as a sequence of patches and uses self-attention mechanism.
- Vision Transformers enable global context learning, allowing them to capture structural inconsistencies, effectively detect AI-generated images, and model relationships between objects more accurately.
- The Vision Transformer works by resizing an image, splitting it into fixed-size patches, flattening them into vectors, converting them into embeddings, and processing them through transformer encoder layers to learn global relationships.

3. DEVELOPMENT OF DEEP LEARNING MODEL (ViT)

Working of ViT

- The input image is divided into small patches (e.g., 16×16), and each patch is flattened and converted into a vector representation called a patch embedding.
- Positional encoding is added to these embeddings so the model can understand the spatial order and location of each patch.
- A special learnable **CLS token** is appended to the patch sequence to aggregate information for final image classification.
- The sequence is passed through transformer encoder blocks consisting of Multi-Head Self-Attention, Feed Forward Networks, Layer Normalization, and Residual Connections to learn rich feature representations.
- Through the self-attention mechanism, each patch attends to all other patches, enabling the model to capture global relationships, long-range dependencies, and overall image context.

3. DEVELOPMENT OF DEEP LEARNING MODEL (ViT)



4. Explainable AI (GRAD-CAM & SHAP)

- Explainable AI (XAI) is used to understand why an image is classified as Real or Fake.
- It improves transparency, trust, and interpretability of the system.
- ImageGuard uses:
  - **Grad-CAM:** Highlights important image regions influencing the CNN (EfficientNet) prediction.
  - **ViT Patch Visualization:** Shows important image patches influencing the Vision Transformer decision.
  - **SHAP** (SHapley Additive Explanations): Analyzes feature contributions.

4.1 GRAD-CAM

- Visualize regions of the image that influence the CNN's decision.
- Helps identify suspicious areas in fake images.

Working

- The input image is processed by the EfficientNet CNN model, which predicts whether the image is **Real or Fake**.
- Grad-CAM analyzes the **final convolutional layer** of the model to determine which image regions contributed most to the prediction.
- These important regions are converted into a **heatmap** and **overlaid on the original image**.

Color Interpretation

- Red regions: Highest influence on the model's decision
- Yellow/Green regions: Moderate influence
- Blue regions: Low influence

4.2 Vision Transformer (ViT) Patch Visualization

- Vision Transformers divide an image into small patches.
- Patch visualization highlights which patches of the image influenced the model's prediction.

Working

- The input image is divided into **fixed-size patches**.
- Each patch is processed by the **Vision Transformer model**, which analyzes relationships between image regions.
- The model calculates **importance scores for each patch** based on its contribution to the prediction.
- Important patches are highlighted as a **grid overlay on the original image**.

Color Interpretation

- **Red patches:** Critical patches with highest importance
- **Orange patches:** High importance regions
- **Yellow patches:** Moderate importance regions
- Other patches are less influential and not highlighted.

4.3 SHAP

- Used to measure how different visual features influence the model's prediction.
- Helps identify which image characteristics support Real or Fake classification.
- Enables comparison of decision behavior between the CNN and ViT models.
- Image Features Used by SHAP include Blurriness level, Edge sharpness,Texture consistency etc.

SHAP Explanation Process

- **Feature Extraction:** Visual features are extracted from the input image.
- **Model Evaluation:** The CNN and ViT predictions are analyzed using these extracted features.
- **Contribution Calculation:** SHAP assigns an importance value to each feature based on its influence on the prediction.

Interpretation of SHAP Values

- Higher SHAP value → stronger influence on the prediction
- Lower SHAP value → weaker influence
- Important features are then ranked and visualized to explain the model's decision.

4.3 SHAP

SHAP for CNN

- Focuses on local texture-based features.
- Helps analyze features such as:
  - Blurriness levels
  - Edge sharpness
  - Texture consistency
- Explains how local visual details influence the model's decision.

SHAP for ViT

- Focuses on global image patterns and patch-level relationships.
- Helps analyze features such as:
  - Shape consistency
  - Structural patterns
  - Lighting and shading distribution
- Effective at identifying global inconsistencies often present in AI-generated images.

5. PRODUCT DESCRIPTION ANALYSIS

The Weight Scoring Algorithm

The system evaluates each listing using two metrics:

- **Match Score** – indicates how well the image matches the description
- **Risk Score** – indicates the level of potential mismatch or fraud

The algorithm starts with a neutral Match Score of 0.5 and a Risk Score of 0. It then adjusts these based on weighted priorities:

- **Category Match (40% Weight)**: The most critical check. A match adds **+0.3** to the score. A mismatch (Bait & Switch) penalizes the score by **-0.4** and adds **+40** to the Risk Score.
- **Color Consistency (30% Weight)**: Compares description keywords against HSV-mapped color clusters. An exact match adds **+0.25**, while a mismatch adds **+25** to the Risk Score.
- **Trust Signals (30% Weight)**: The presence of professional metadata like **Brand (+0.1)**, **Price (+0.1)**, and **Size (+0.1)** rewards the listing for being detailed and transparent.

5. PRODUCT DESCRIPTION ANALYSIS

- This module serves as the **Multimodal Verification Layer** of the ImageGuard system. Its purpose is to verify whether the product description matches the product image.
- The browser extension automatically scrapes the product title and description from the marketplace. This information is sent to the **FastAPI** backend, where features are extracted and a **match score** is calculated.

Technical Implementation Pipeline

The system performs analysis in three stages:

1. **Feature Extraction (NLP)**: Regular Expressions (Regex) and keyword mapping are used to extract key attributes from the product description, such as: Brand, Color, Size, Price, and Condition.
2. **Visual-Text Cross-Referencing**: These extracted text attributes are compared against the live outputs of the YOLOv8 classifier and K-Means Color Analysis.
3. **Similarity Logic**: The system applies a Weighted Scoring Algorithm where critical mismatches—such as a category conflict (text says "Watch" but image shows "Shoe")—trigger the highest risk alerts.

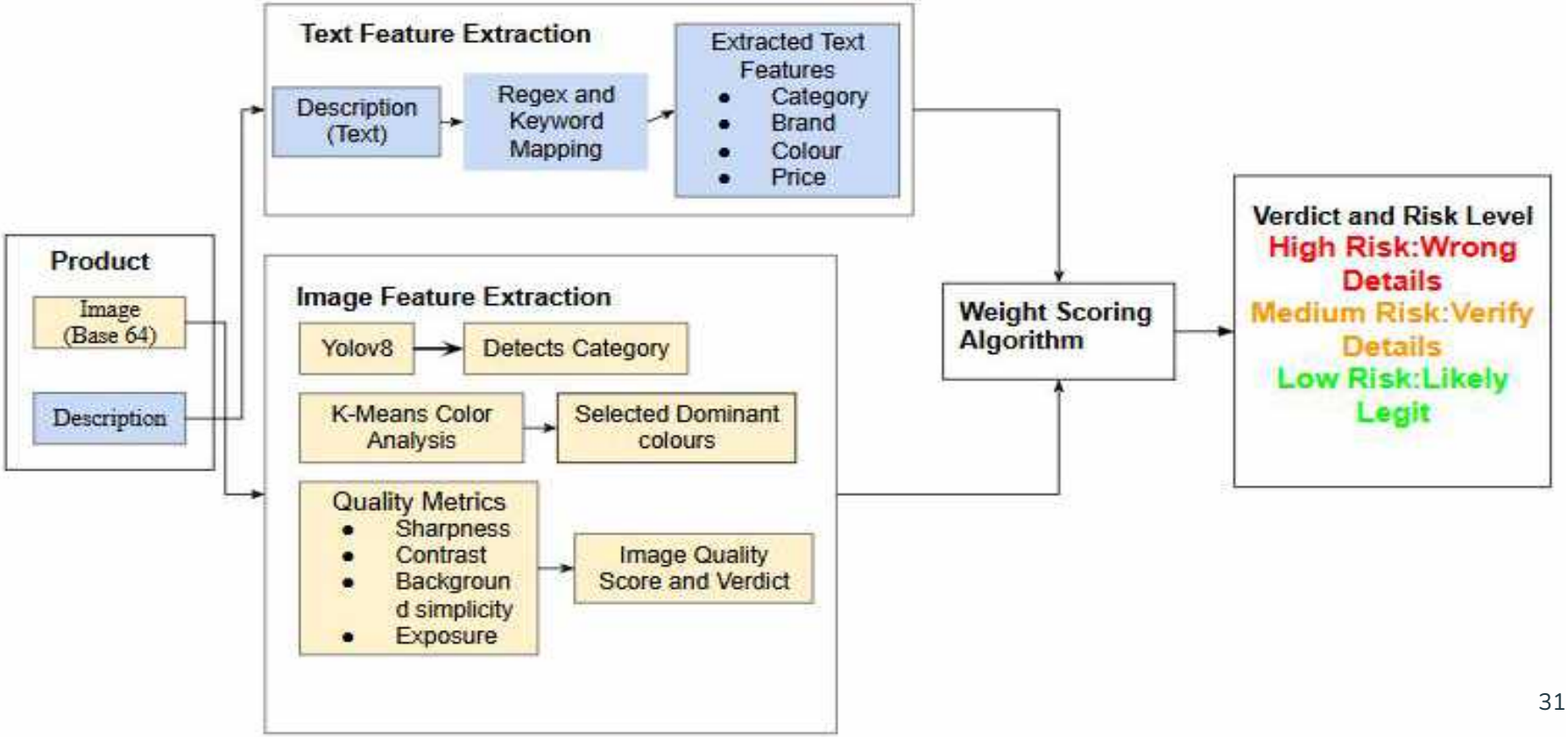
5. PRODUCT DESCRIPTION ANALYSIS

Risk Assessment & Final Verdicts

The cumulative **Risk Score** determines the final instruction given to the user:

Risk Level	Risk Score	Final Verdict
High	60+	POTENTIAL SCAM
Medium	35 - 59	VERIFY CAREFULLY
Low	< 35	LIKELY LEGITIMATE

5. PRODUCT DESCRIPTION ANALYSIS



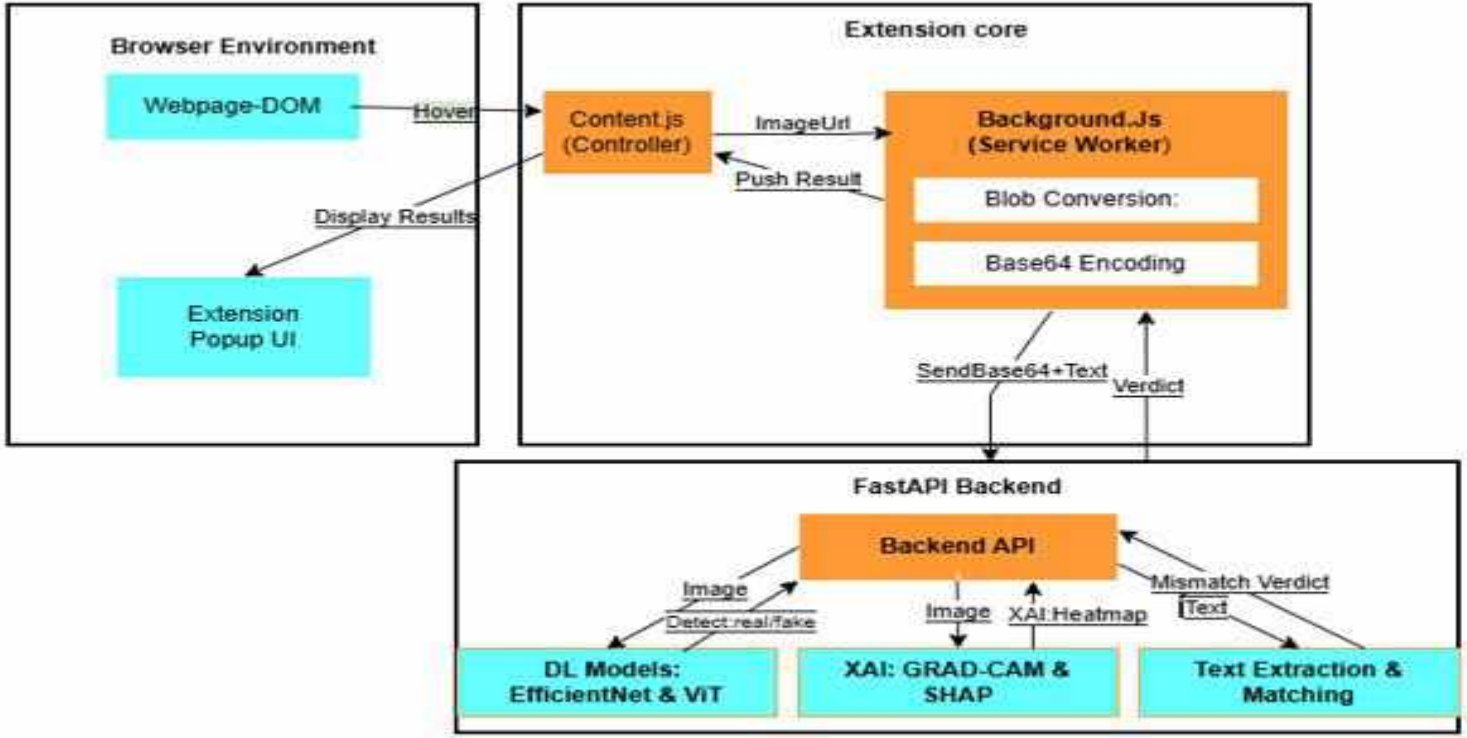
6. BROWSER EXTENSION

- Developed to enable real-time fake image detection on online marketplaces.
- Allows users to check image authenticity directly during browsing.
- Works without modifying the existing marketplace infrastructure.

Working Process

- Product images are extracted from the webpage
- Images are preprocessed within the extension
- Preprocessed images are sent to the backend via REST-based API
- EfficientNet and ViT models perform inference
- System returns:
  - Classification result (Real / Fake)
  - Confidence score
  - Explainable AI (XAI) results

6. BROWSER EXTENSION



Results

CNN

Dataset Information:

Training samples: 13456
Validation samples: 3214
Test samples: 3217
Classes: Fake, Real

Confusion Matrix:

[1490 10]
[11 1706]

Training vs Validation Performance

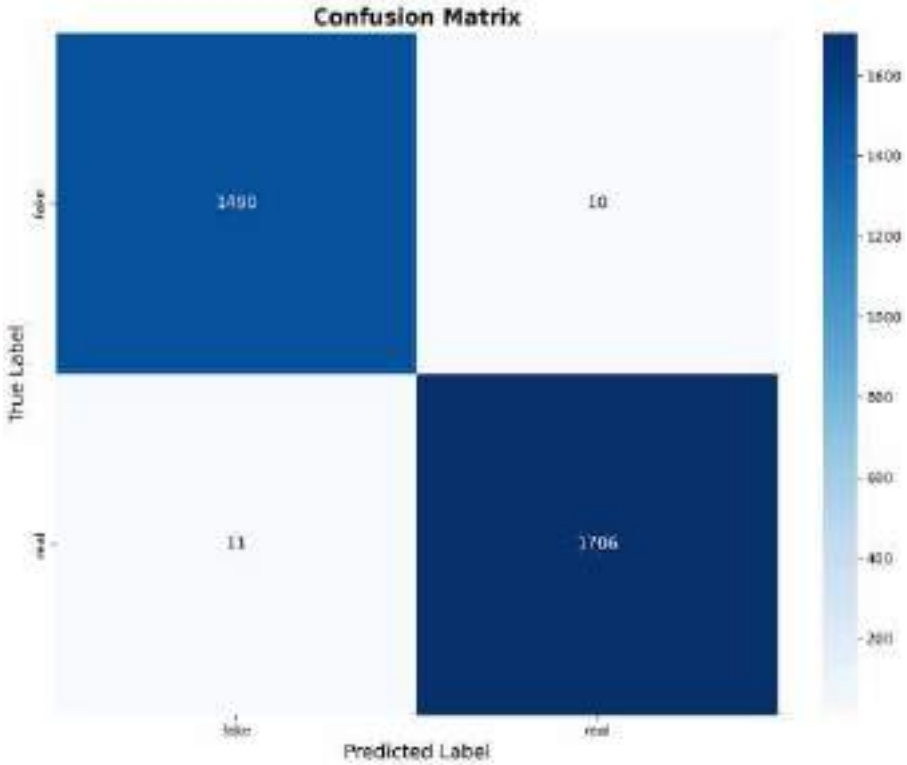
Best Validation Accuracy: 99.25%
Best Validation Loss: 0.0315
Total Training Time: 55.7 minutes

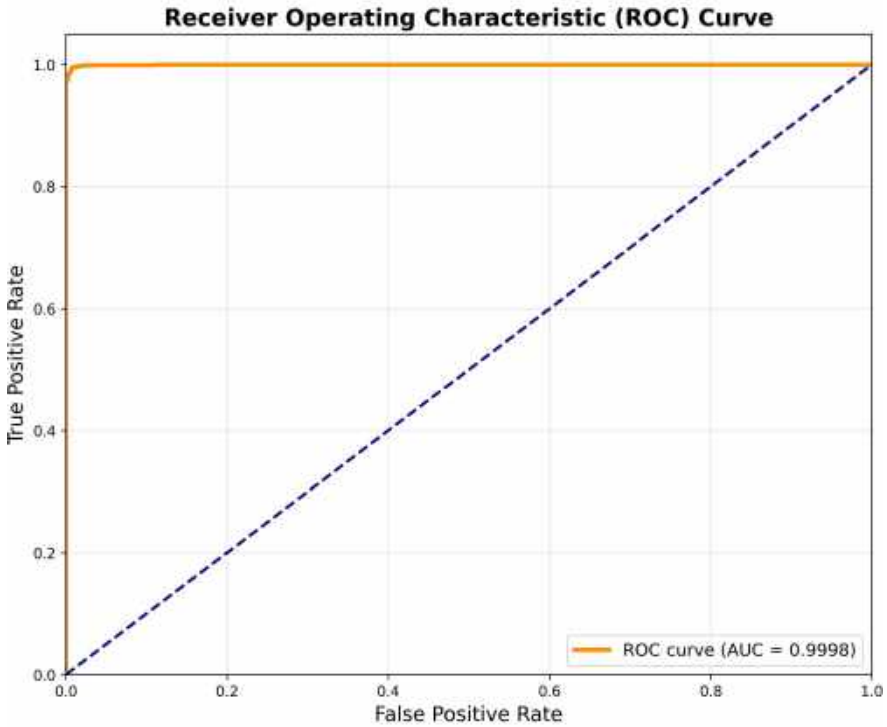
Class	Precision	Recall	F1-Score	Support
Fake	0.9927	0.9933	0.9930	1500
Real	0.9942	0.9936	0.9939	1717

Test Results:

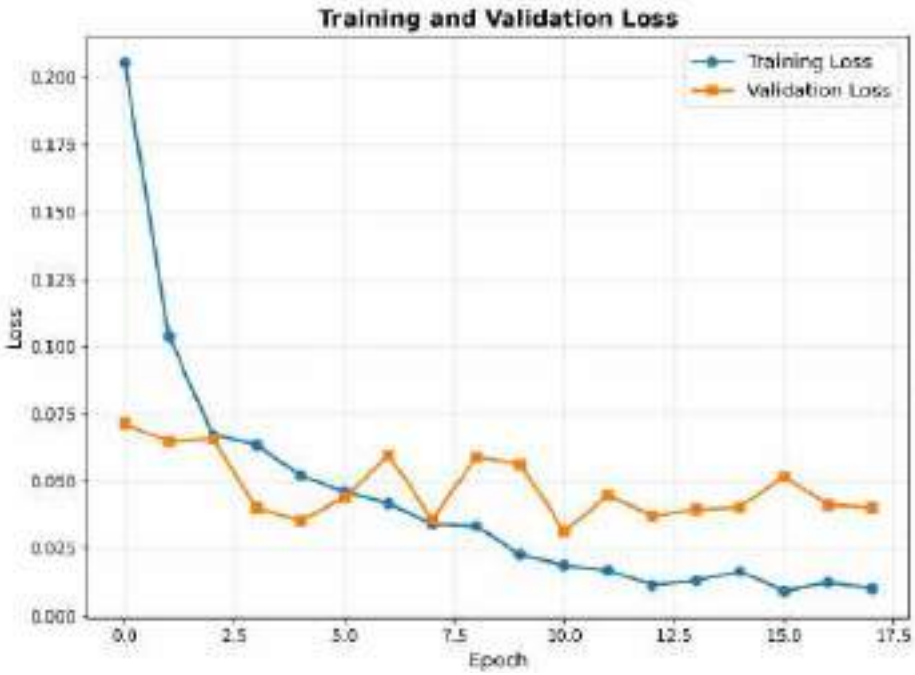
Test Accuracy: 99.3472%
ROC AUC Score: 0.9998

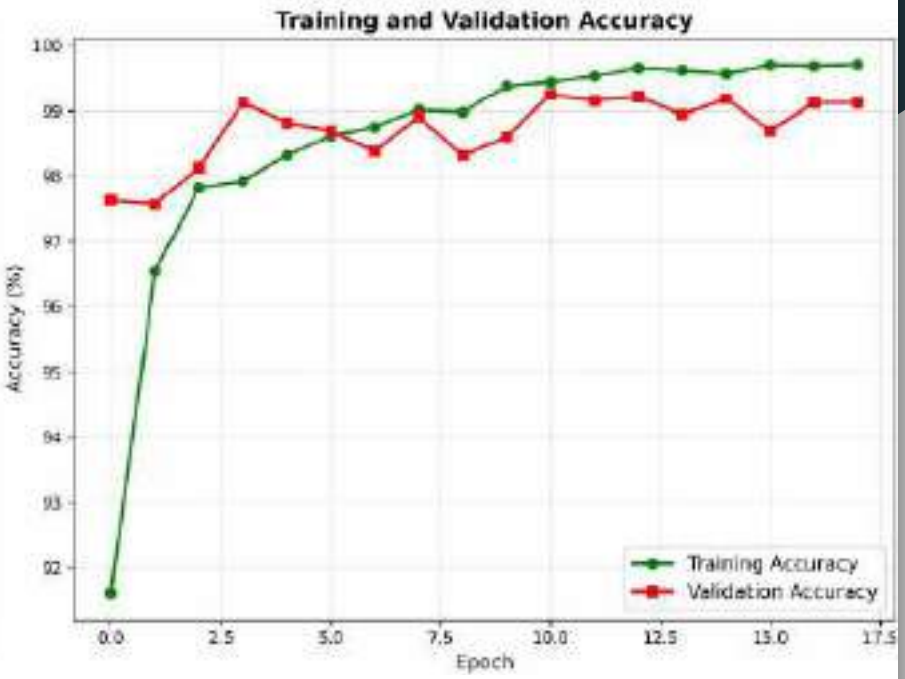
CNN


Confusion Matrix
1490
10
11
1706
True Label
Predicted Label
fake
real
1600
1400
1200
1000
800
600
400
200


Receiver Operating Characteristic (ROC) Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (AUC = 0.9998)

CNN


Training and Validation Loss
Training Loss
Validation Loss
Loss
Epoch
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5


Training and Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy (%)
Epoch
100
99
98
97
96
95
94
93
92
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5

ViT

Dataset Information:

Training samples: 13456
Validation samples: 3214
Test samples: 3217
Classes: Fake, Real

Confusion Matrix:

[1478 22]
[10 1707]

Training vs Validation Performance

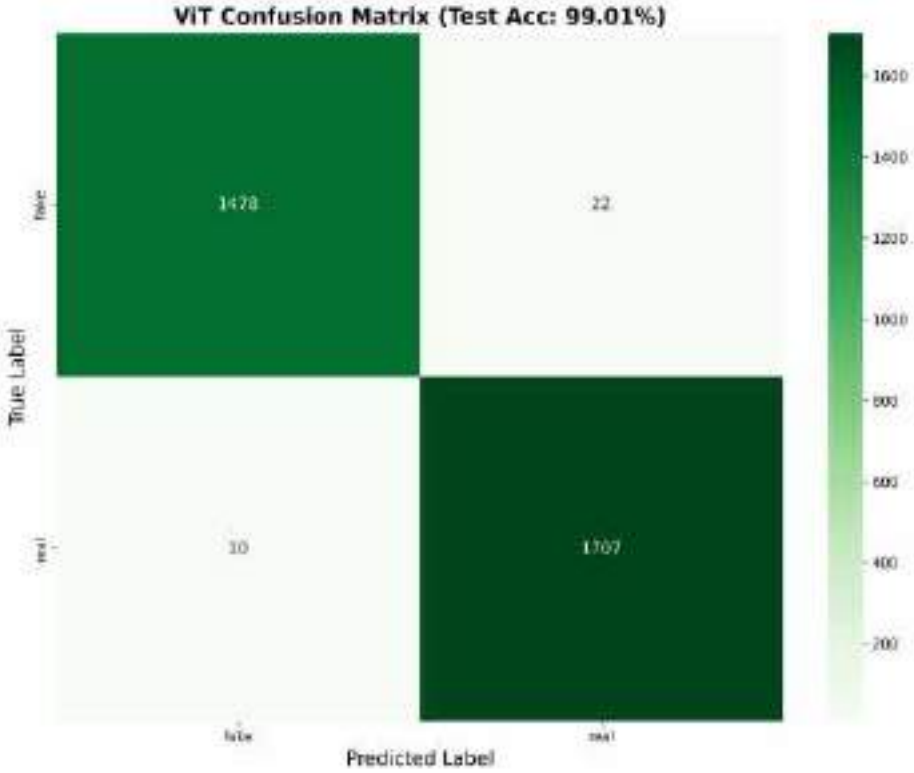
Best Validation Accuracy: 98.66%
Best Validation Loss: 0.0694
Total Training Time: 238.8 minutes

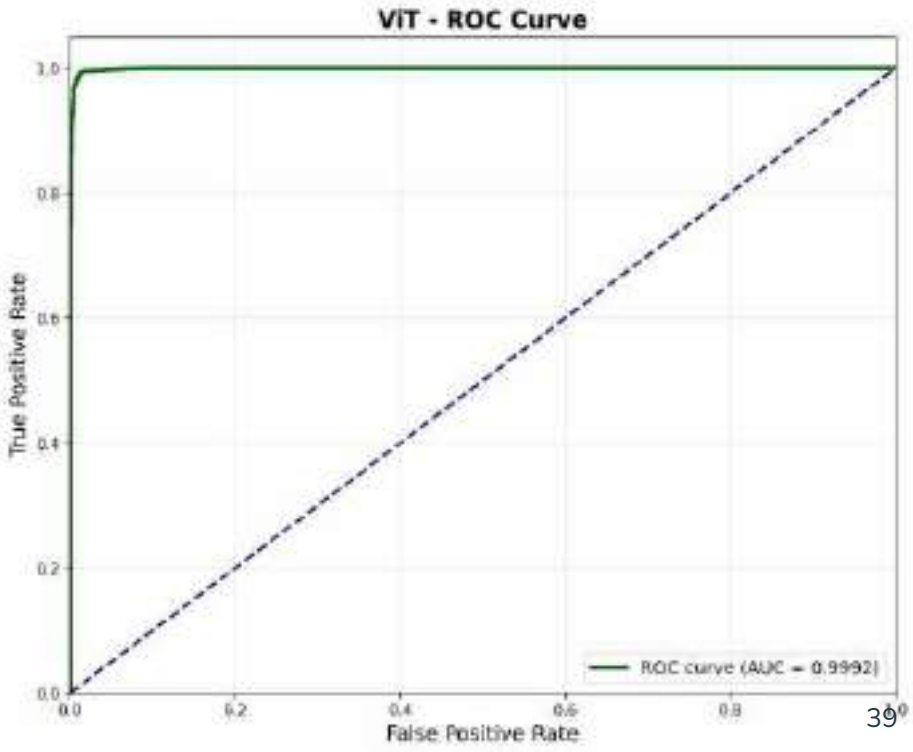
Class	Precision	Recall	F1-Score	Support
Fake	0.9933	0.9853	0.9893	1500
Real	0.9873	0.9942	0.9907	1717

Test Results:

Test Accuracy: 99.0053%
ROC AUC Score: 0.9992

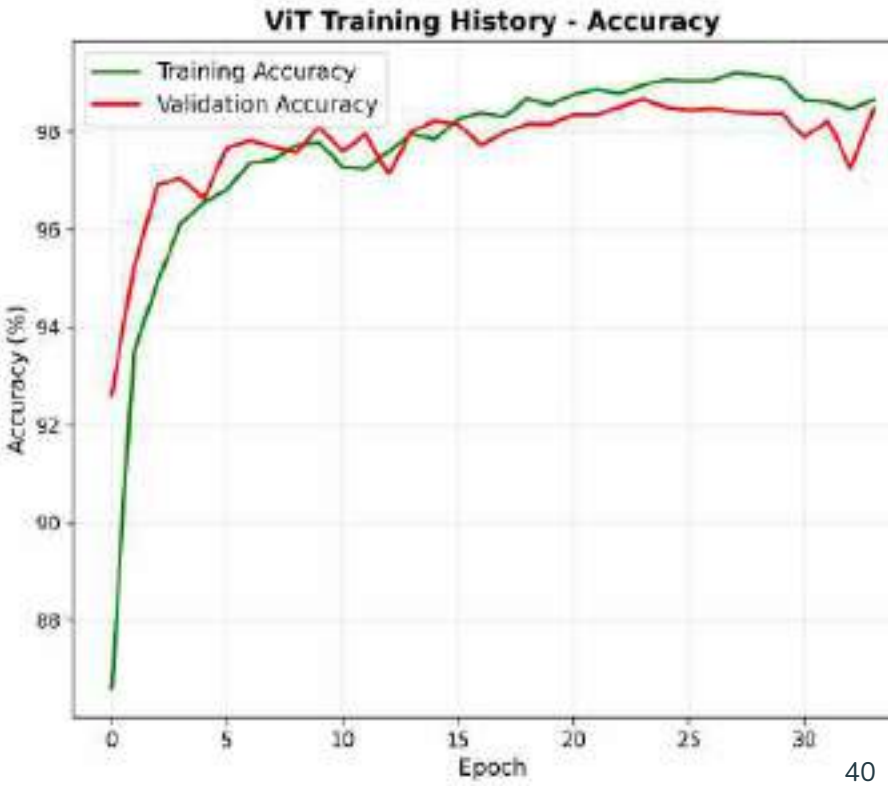
ViT


ViT Confusion Matrix (Test Acc: 99.01%)
1478
22
10
1707
True Label
Predicted Label
1600
1400
1200
1000
800
600
400
200


ViT - ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC curve (AUC = 0.9992)

ViT

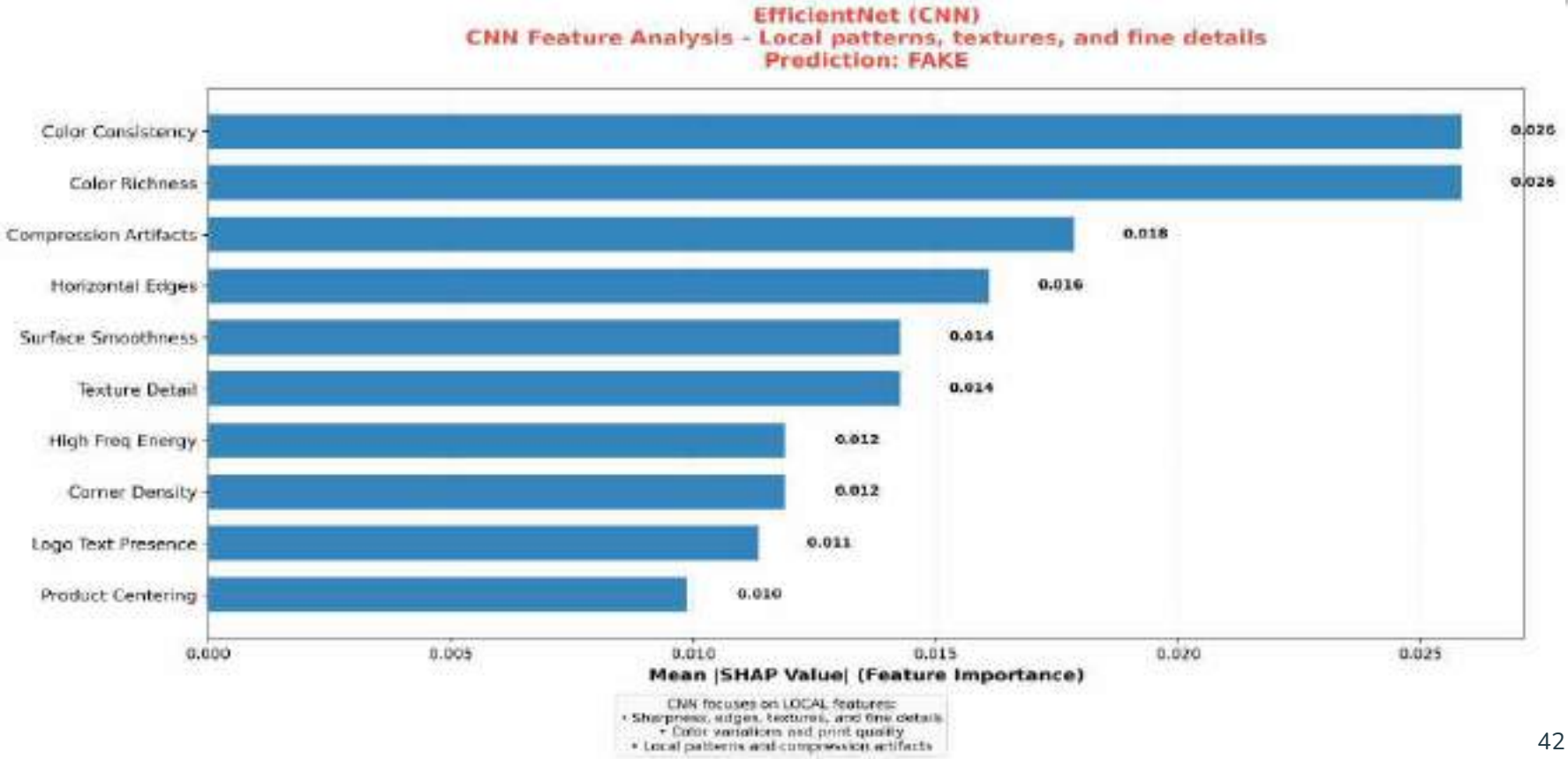
ViT Training History - Loss
Training Loss
Validation Loss
Loss
0.30
0.25
0.20
0.15
0.10
0.05
0
5
10
15
20
25
30
Epoch


ViT Training History - Accuracy
Training Accuracy
Validation Accuracy
Accuracy (%)
98
96
94
92
90
88
0
5
10
15
20
25
30
Epoch

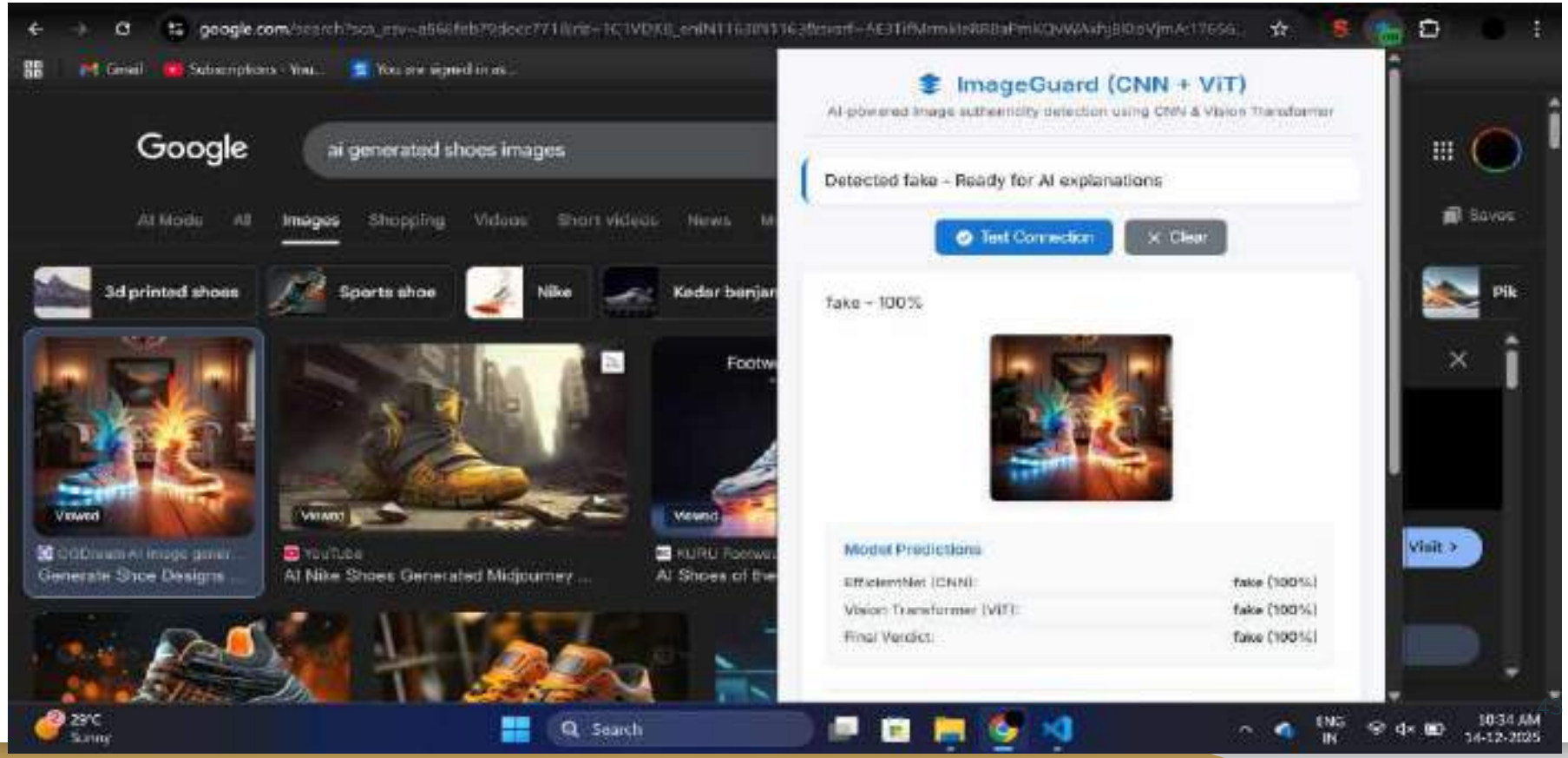
GRAD-CAM

Original Image
True Label: fake
EfficientNet (CNN)
Prediction: fake (100.0%)
✓ CORRECT
CNN (EfficientNet) uses Grad-CAM
• Shows which regions influenced prediction
• RED: High influence areas
• BLUE: Low influence areas
• Analyzes local patterns and textures
• For 'fake' prediction, focused on specific features
1.0
0.8
0.6
0.4
0.2
0.0
Feature Importance

SHAP


EfficientNet (CNN)
CNN Feature Analysis - Local patterns, textures, and fine details
Prediction: FAKE
Color Consistency
Color Richness
Compression Artifacts
Horizontal Edges
Surface Smoothness
Texture Detail
High Freq Energy
Corner Density
Logo Text Presence
Product Centering
0.026
0.026
0.018
0.016
0.014
0.014
0.012
0.012
0.011
0.010
0.000
0.005
0.010
0.015
0.020
0.025
Mean |SHAP Value| (Feature Importance)
CNN focuses on LOCAL features:
• Sharpness, edges, textures, and fine details
• Color variations and print quality
• Local patterns and compression artifacts

BROWSER EXTENSION



RESULTS-CNN (Real)

```
(venv_pytorch) PS D:\project> & D:/project/venv_pytorch/Scripts/Activate.ps1
>> python D:\project\predict_new_image.py
Using device: cuda

Loading EfficientNet-B2 model...
✓ EfficientNet-B2 loaded successfully
  Model accuracy: 99.72677595628416
  Classes: ['fake', 'real']

======================================================================
EFFICIENTNET-B2 IMAGE PREDICTION SYSTEM
======================================================================
Model: EfficientNet-B2 (99.74% test accuracy)
Device: cuda
======================================================================

==================================================
MAIN MENU
==================================================
1. Predict single image
2. Batch predict from folder
3. Show prediction statistics
4. Reset statistics
5. Test on sample images
6. Exit
==================================================

Select option (1-6): 1
Enter image path: C:\Users\ASUS\Downloads\image1.jpg
```

CNN test image (Real):

```
============================================================
CNN PREDICTION RESULTS
============================================================
Image: image1.jpg

Prediction: REAL
Confidence: 100.00%
Detailed Probabilities:
  Fake: 0.00%
  Real: 100.00%
============================================================
```

RESULTS-CNN (Fake)

```
(venv_pytorch) PS D:\project> & D:/project/venv_pytorch/Scripts/Activate.ps1
>> python D:\project\predict_new_image.py
Using device: cuda

Loading EfficientNet-B2 model...
✓ EfficientNet-B2 loaded successfully
  Model accuracy: 99.72677595628416
  Classes: ['fake', 'real']

================================================================
EFFICIENTNET-B2 IMAGE PREDICTION SYSTEM
================================================================
Model: EfficientNet-B2 (99.74% test accuracy)
Device: cuda
================================================================

================================================
MAIN MENU
================================================
1. Predict single image
2. Batch predict from folder
3. Show prediction statistics
4. Reset statistics
5. Test on sample images
6. Exit
================================================

Select option (1-6): 1
Enter image path: C:\Users\ASUS\Downloads\image.jpg
```

CNN test image (Fake):

```
============================================================
CNN PREDICTION RESULTS
============================================================
Image: image.jpg

Prediction: FAKE
Confidence: 99.99%
Detailed Probabilities:
  Fake: 99.99%
  Real: 0.01%
============================================================
```

RESULTS-ViT (Real)

```
(venv_pytorch) PS D:\project> & D:/project/venv_pytorch/Scripts/Activate.ps1
>> python D:\project\check_vit.py
Using device: cuda
GPU: NVIDIA GeForce RTX 3050 Laptop GPU

Loading Vision Transformer (ViT-Base) model...
✓ Loaded best ViT model
  Best validation accuracy: 97.45%
✓ ViT model loaded and ready for inference
  Device: cuda
  Parameters: 86,327,298

================================================================================
VISION TRANSFORMER (ViT) PREDICTION SYSTEM
================================================================================
Model: ViT-Base Patch16 224
Device: cuda
================================================================================

================================================
ViT MAIN MENU
================================================
1. Predict single image
2. Batch predict from folder
3. Show prediction statistics
4. Reset statistics
5. Test on sample images
6. Exit
================================================

Select option (1-6): 1
Enter image path: C:\Users\ASUS\Downloads\image1.jpg
```

ViT test image (Real):

```
================================================
ViT PREDICTION RESULTS
================================================
Image: image1.jpg
Prediction: real
Confidence: 99.92%
Fake Probability: 0.08%
Real Probability: 99.92%
Processing Time: 299.2 ms
================================================
```

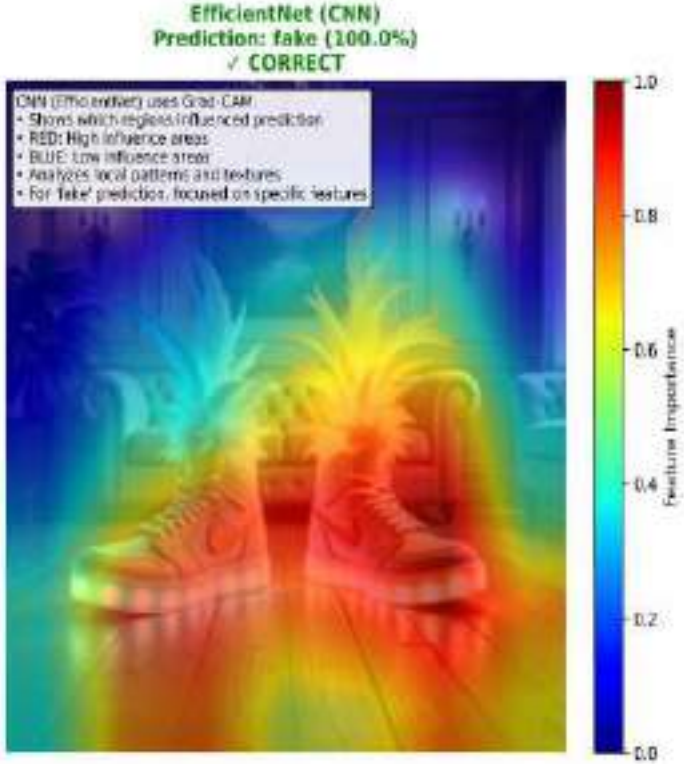
RESULTS-ViT (Fake)

```
(venv_pytorch) PS D:\project> & D:/project/venv_pytorch/Scripts/Activate.ps1
>> python D:\project\check_vit.py
Using device: cuda
GPU: NVIDIA GeForce RTX 3050 Laptop GPU

Loading Vision Transformer (ViT-Base) model...
✓ Loaded best ViT model
  Best validation accuracy: 97.45%
✓ ViT model loaded and ready for inference
  Device: cuda
  Parameters: 86,327,298

================================================================================
VISION TRANSFORMER (ViT) PREDICTION SYSTEM
================================================================================
Model: ViT-Base Patch16 224
Device: cuda
================================================================================

================================================
ViT MAIN MENU
================================================
1. Predict single image
2. Batch predict from folder
3. Show prediction statistics
4. Reset statistics
5. Test on sample images
6. Exit
================================================

Select option (1-6): 1
Enter image path: C:\Users\ASUS\Downloads\image.jpg
```

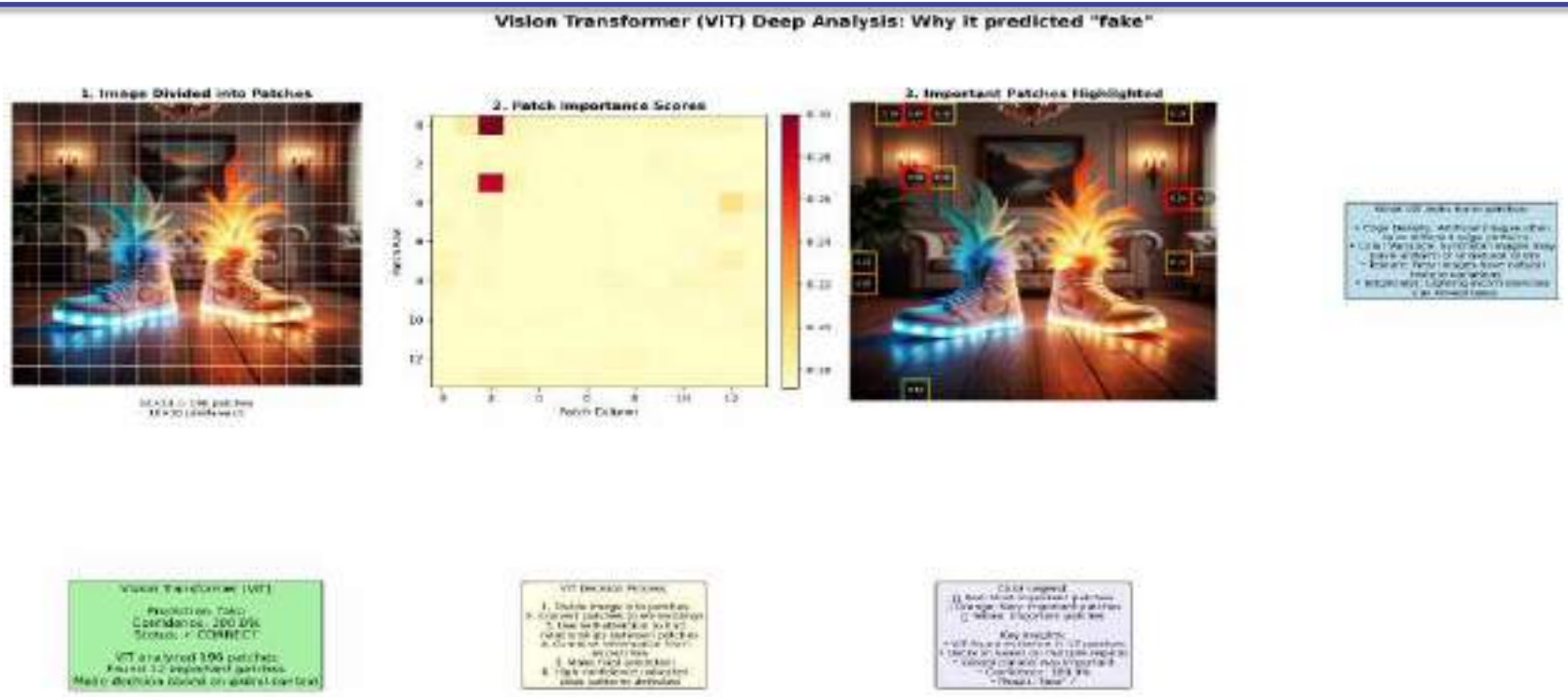
ViT test image (Fake):

```
================================================
ViT PREDICTION RESULTS
================================================
Image: image.jpg
Prediction: fake
Confidence: 99.96%
Fake Probability: 99.96%
Real Probability: 0.04%
Processing Time: 302.2 ms
================================================
```

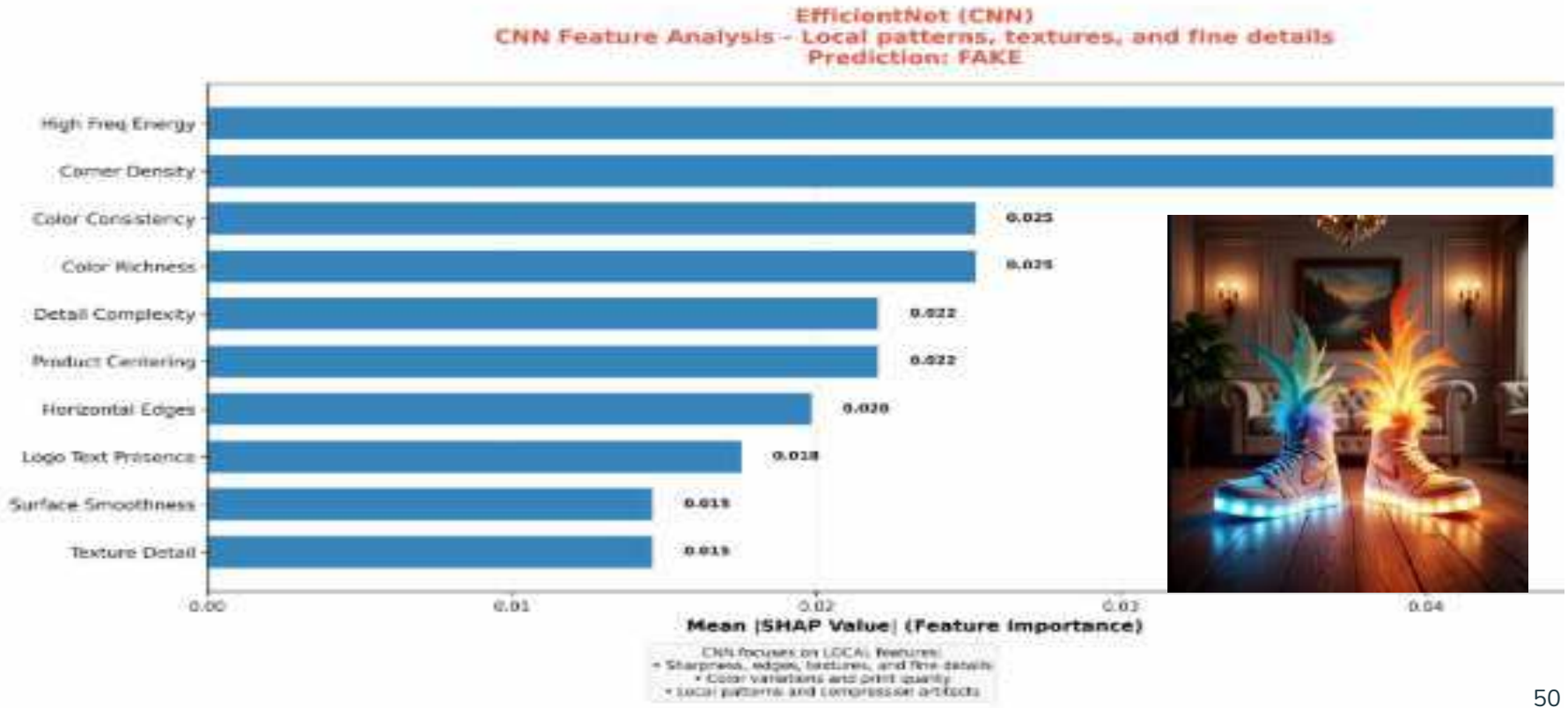
RESULTS-XAI (GRADCAM+CNN)



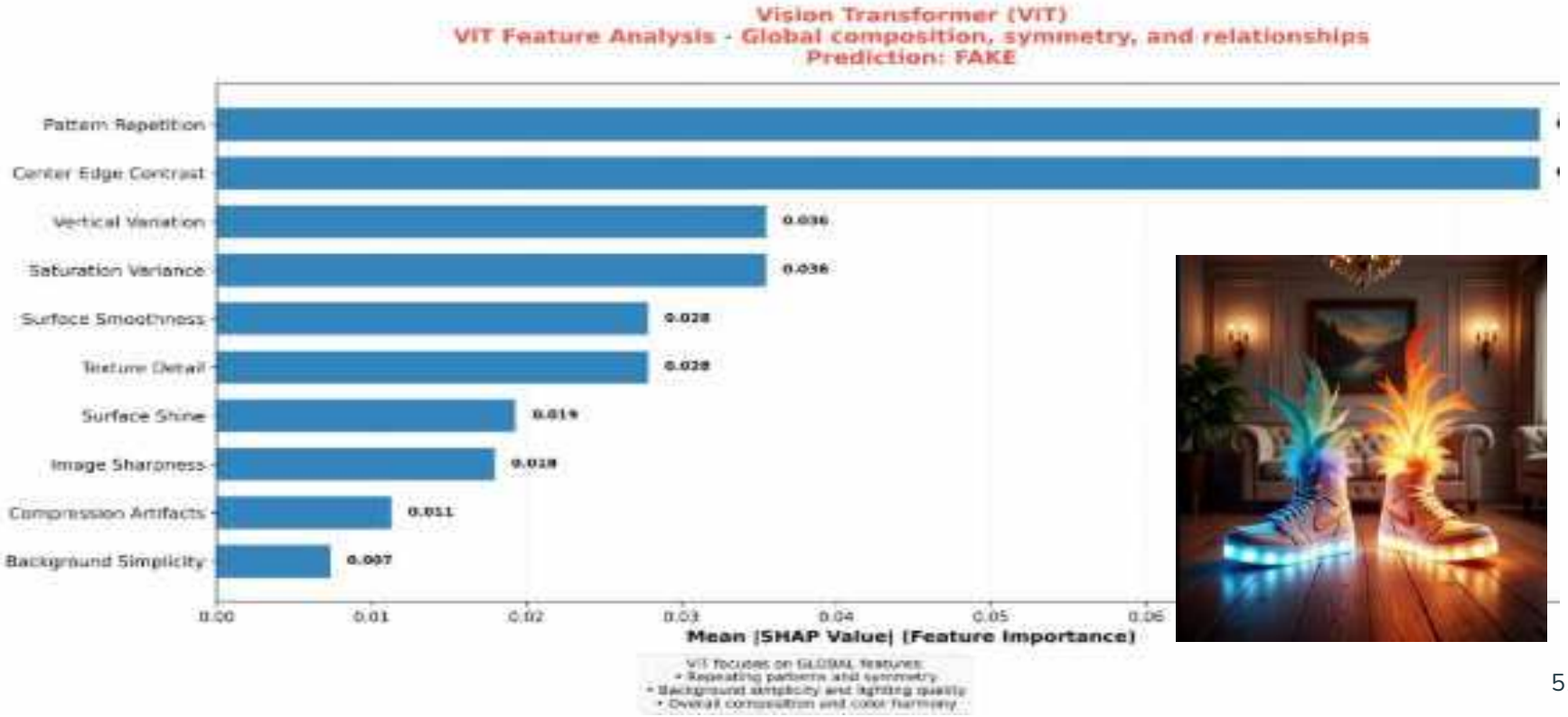
RESULTS-XAI (GRADCAM+ViT)



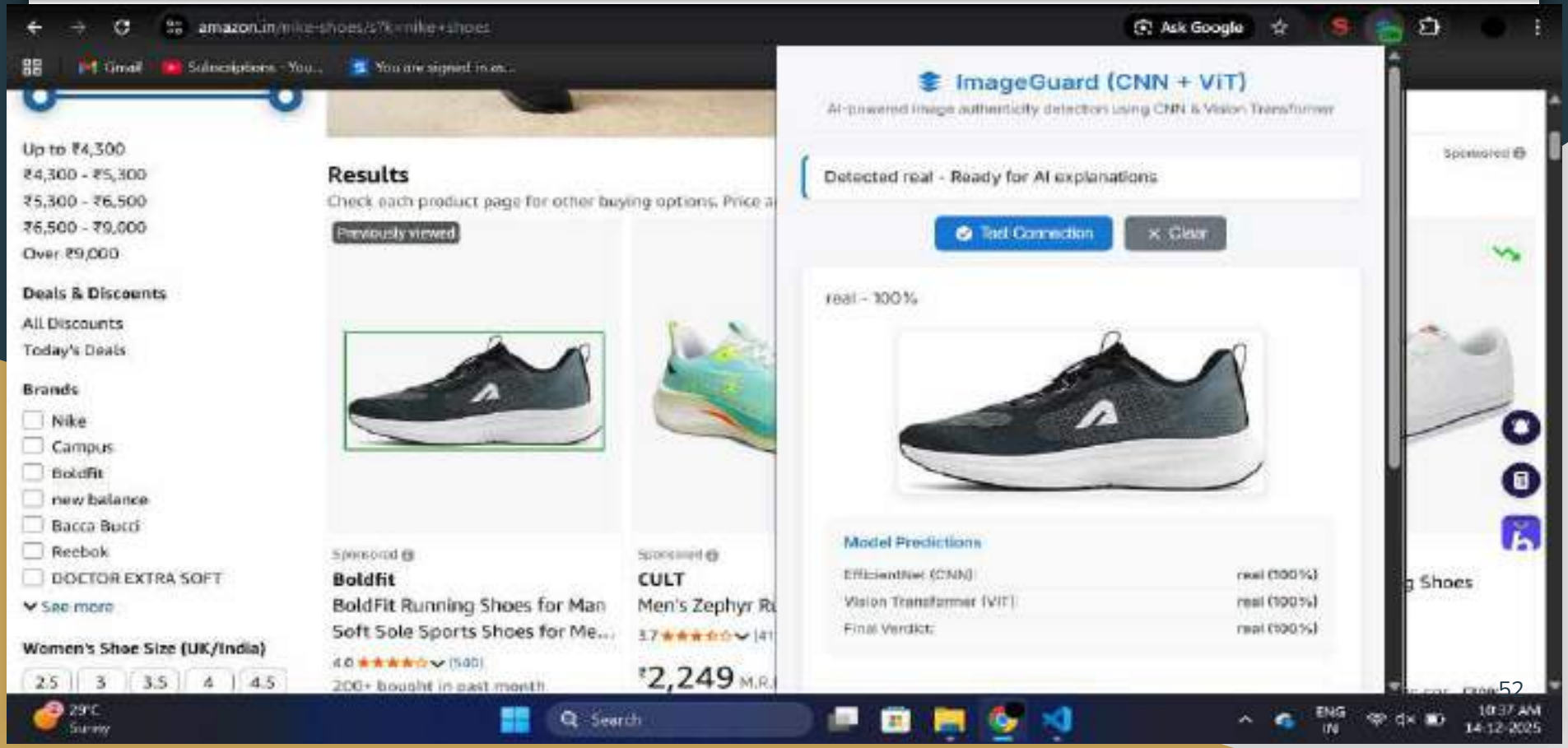
RESULTS-XAI (SHAP+CNN)



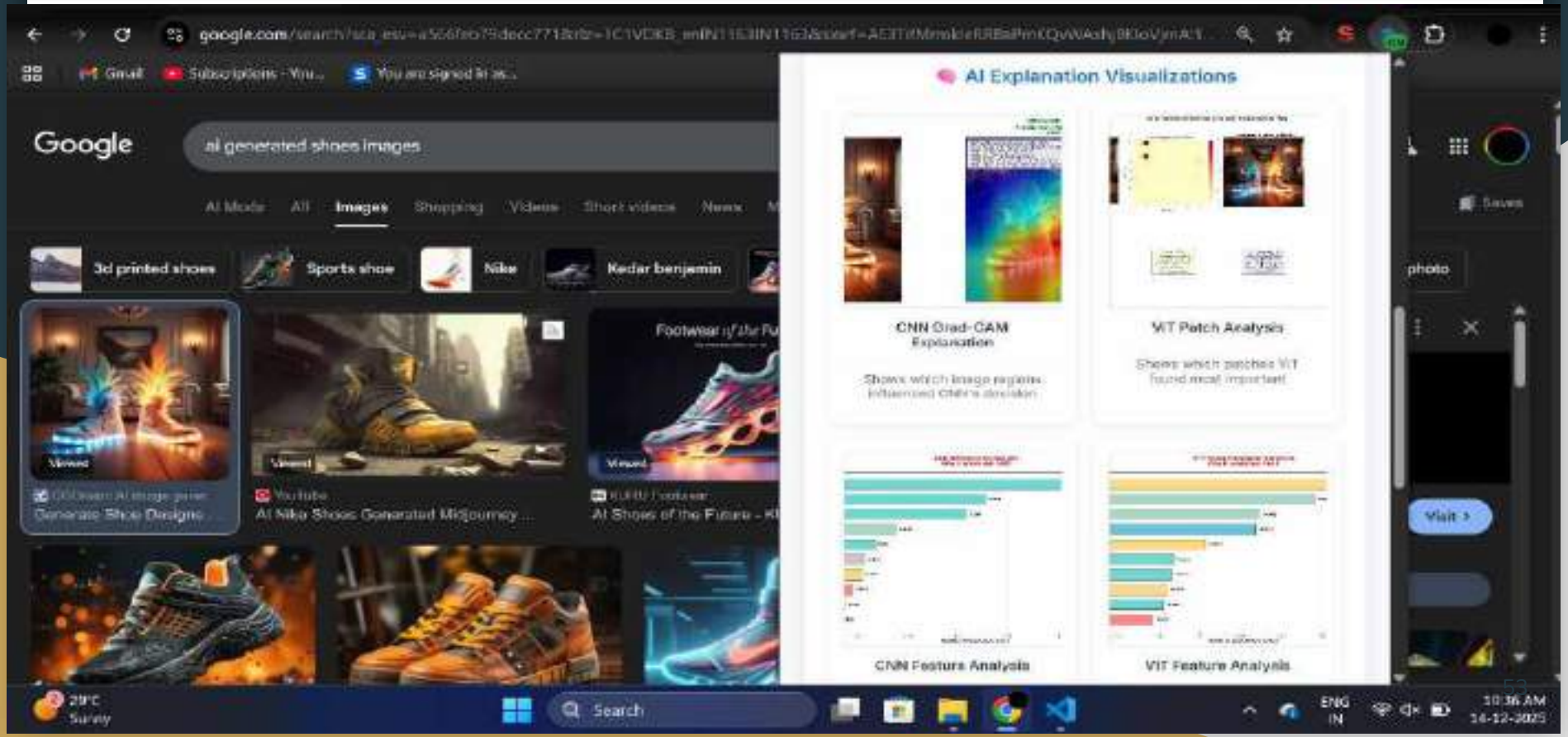
RESULTS-XAI (SHAP+ViT)



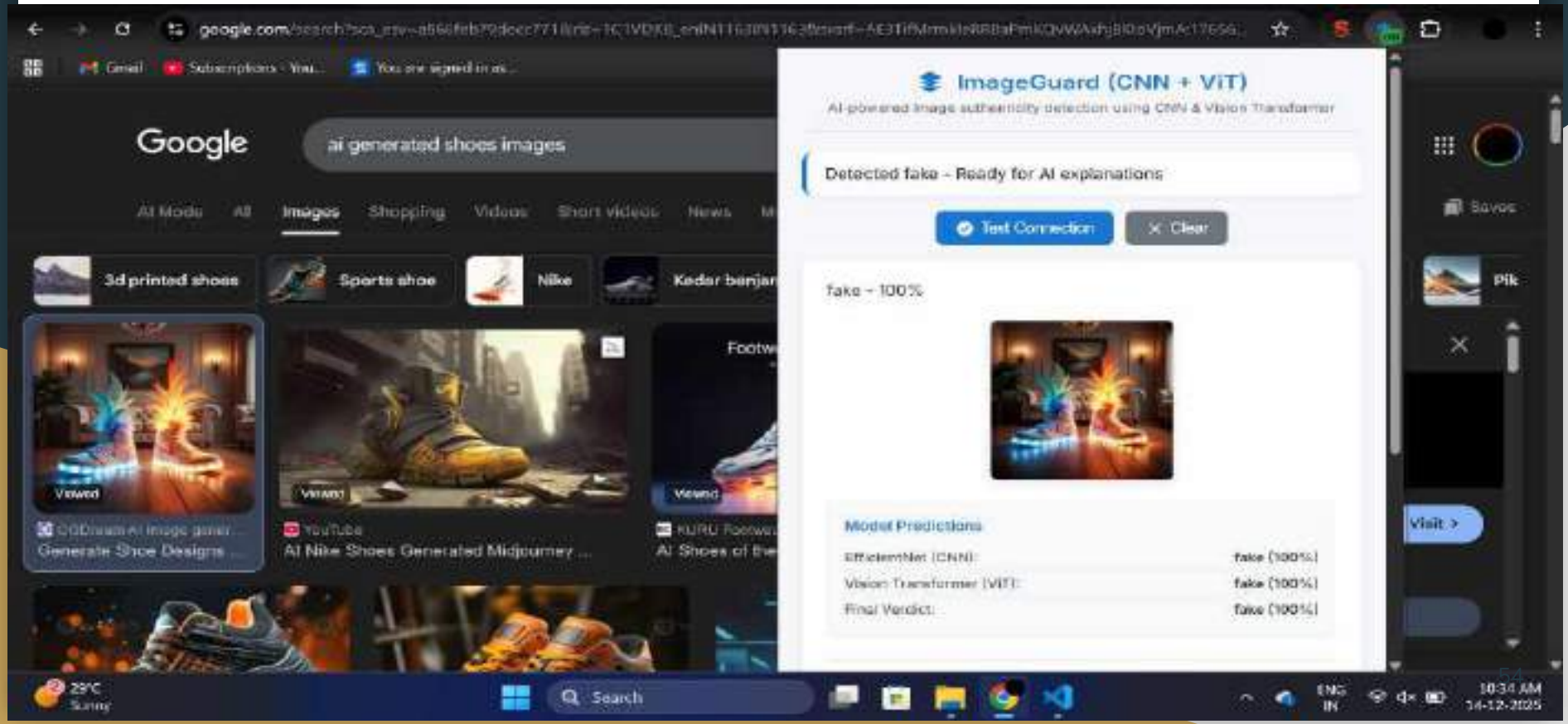
RESULTS-BROWSER EXTENSION (Real)



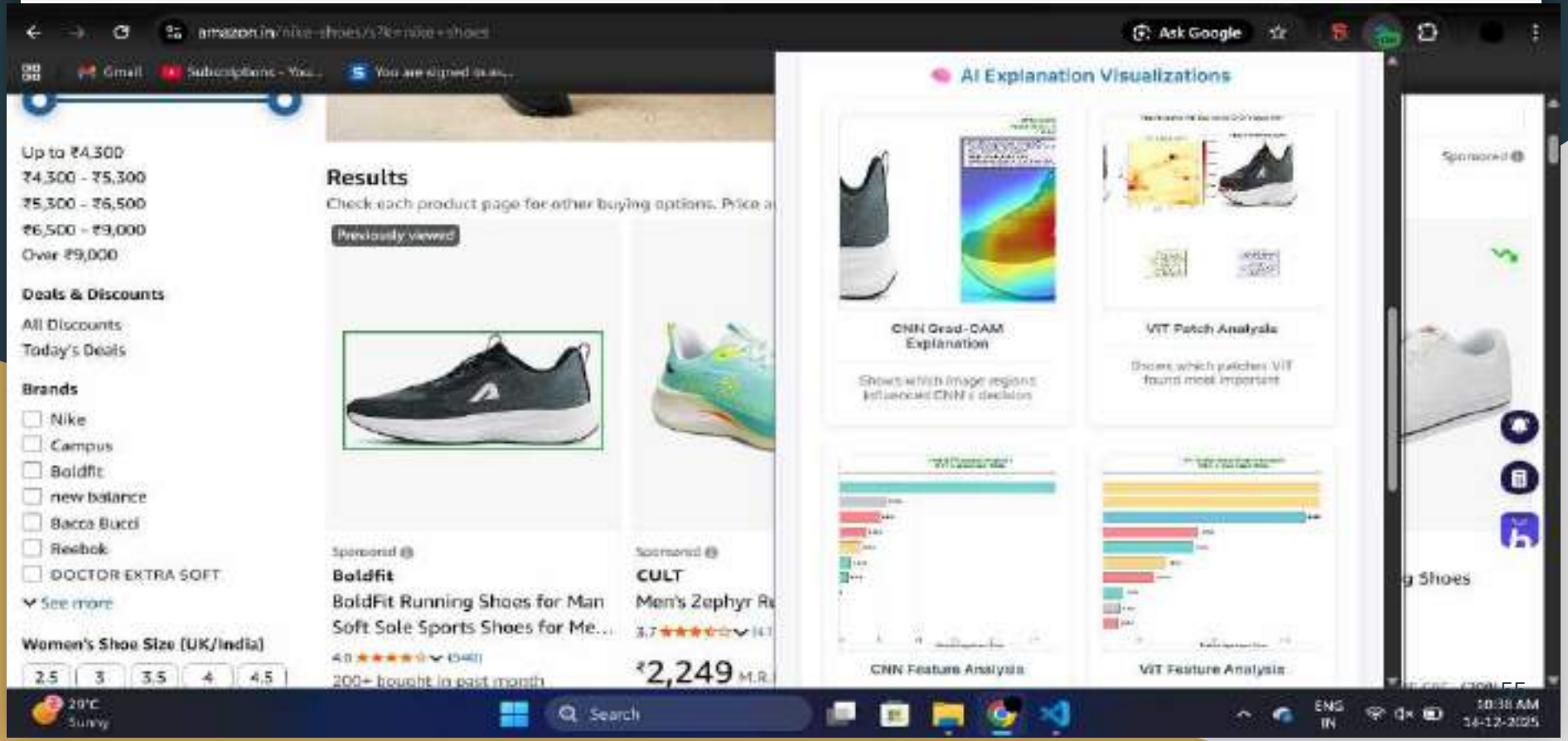
RESULTS-BROWSER EXTENSION (Real)



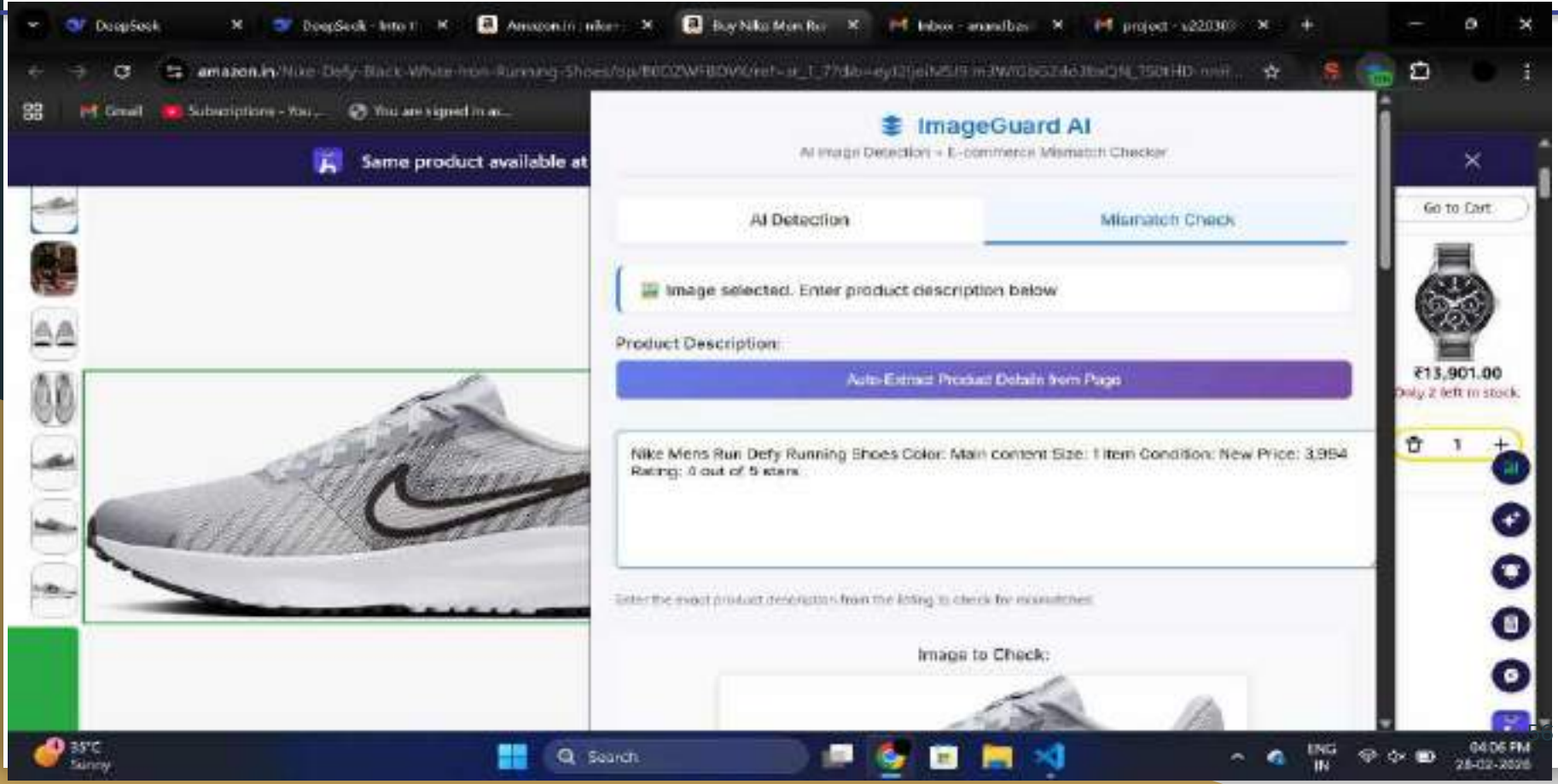
RESULTS-BROWSER EXTENSION (Fake)



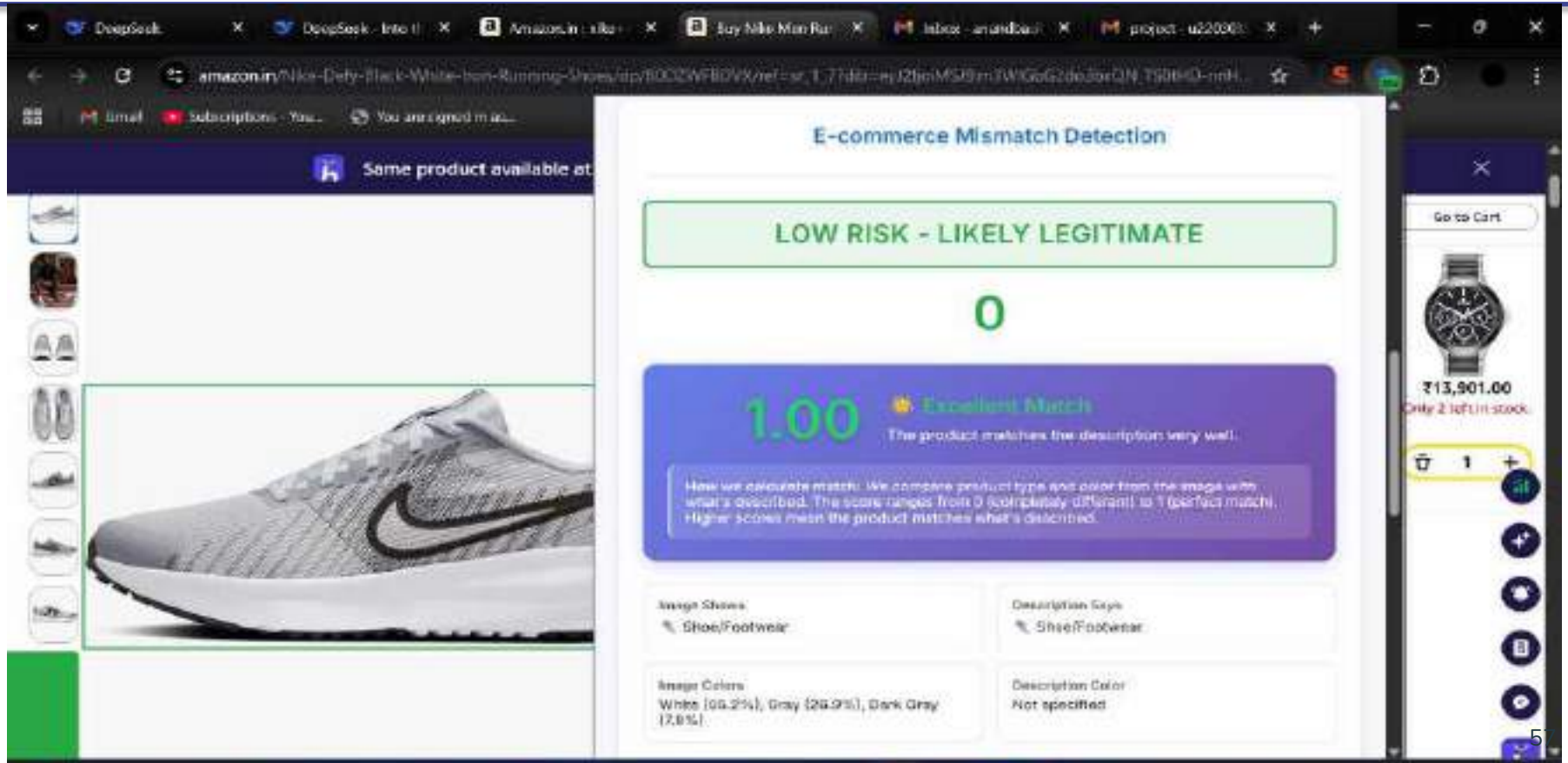
RESULTS-BROWSER EXTENSION (Fake)



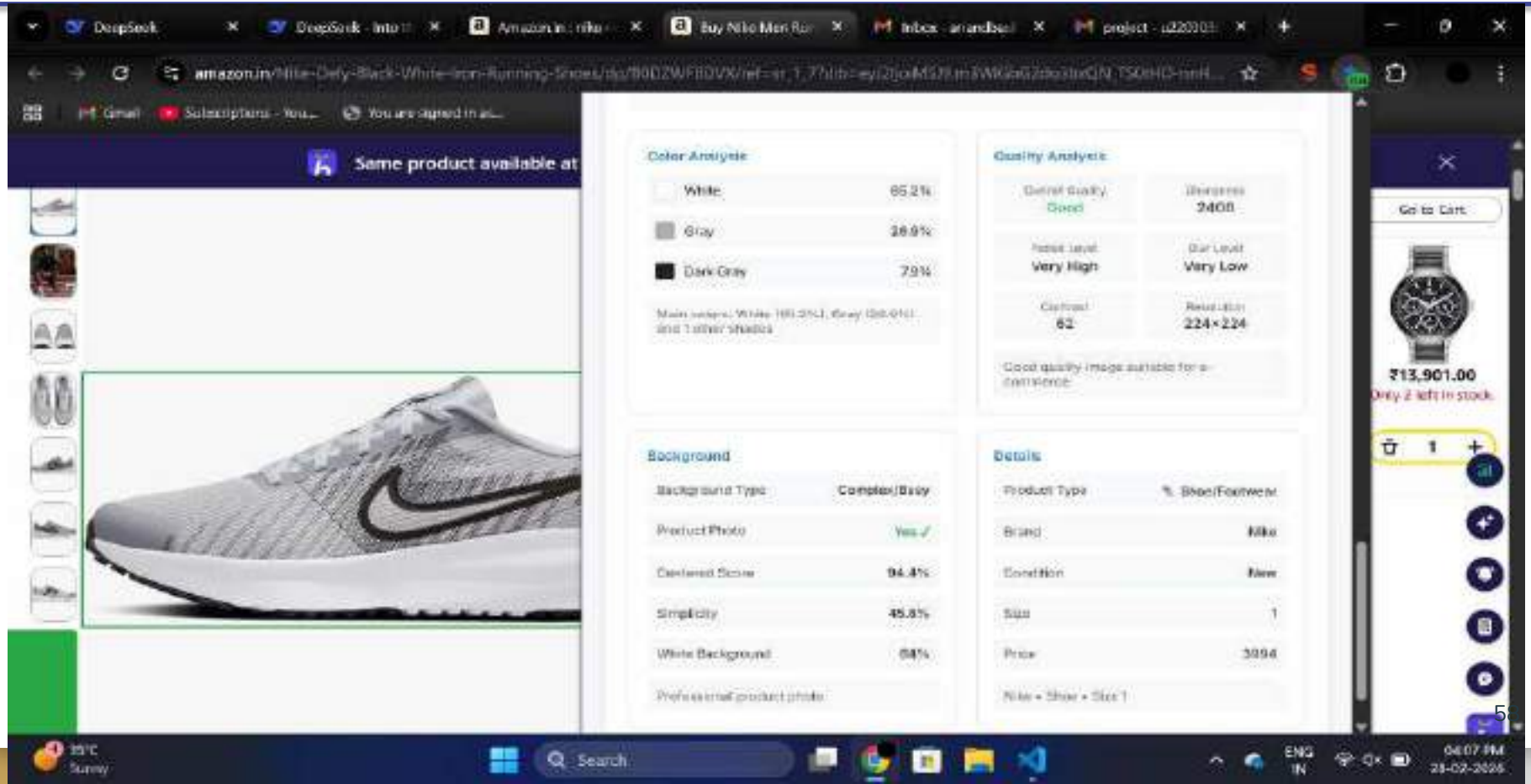
PRODUCT DESCRIPTION ANALYSIS (Real)



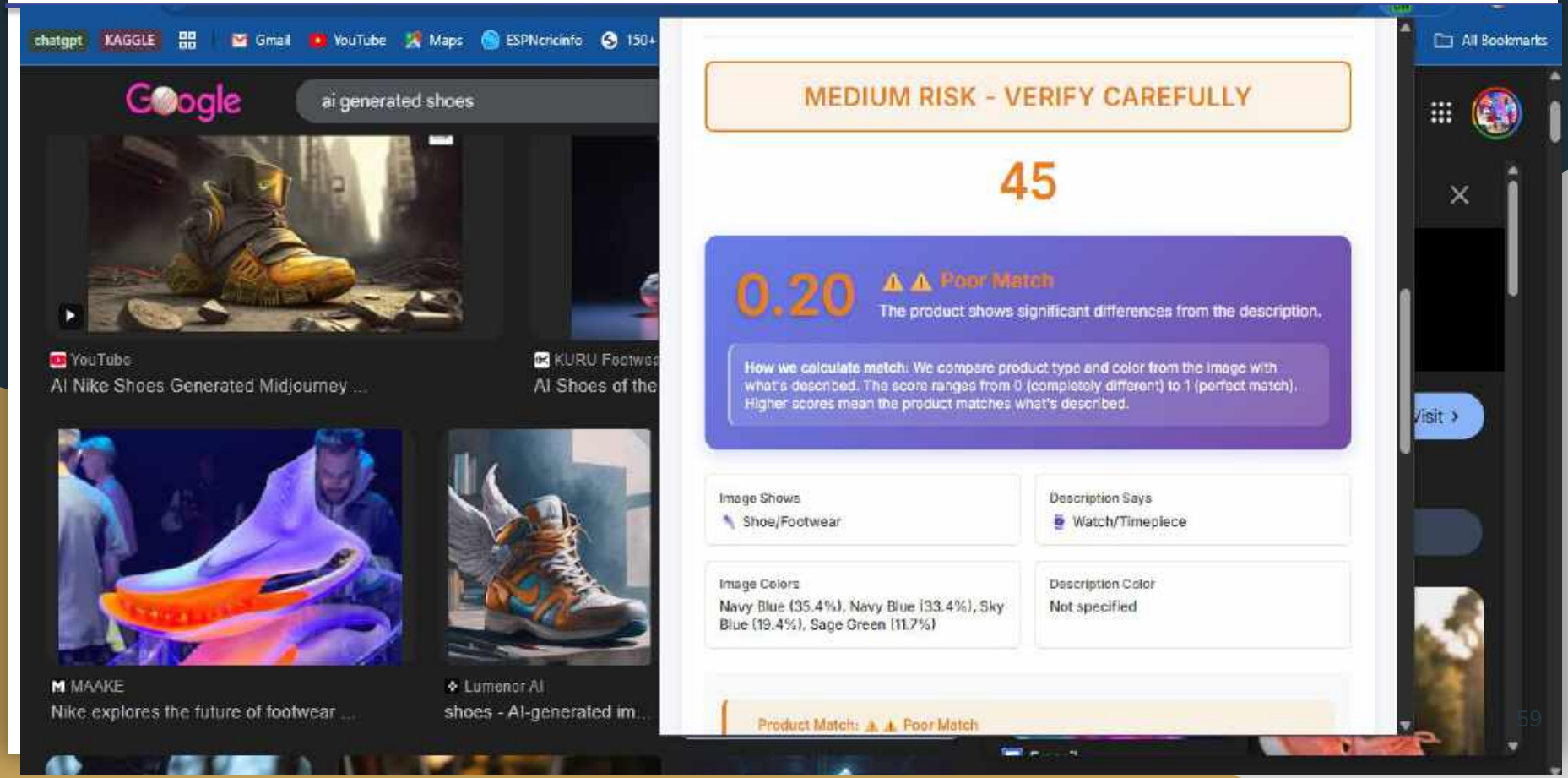
PRODUCT DESCRIPTION ANALYSIS (Real)



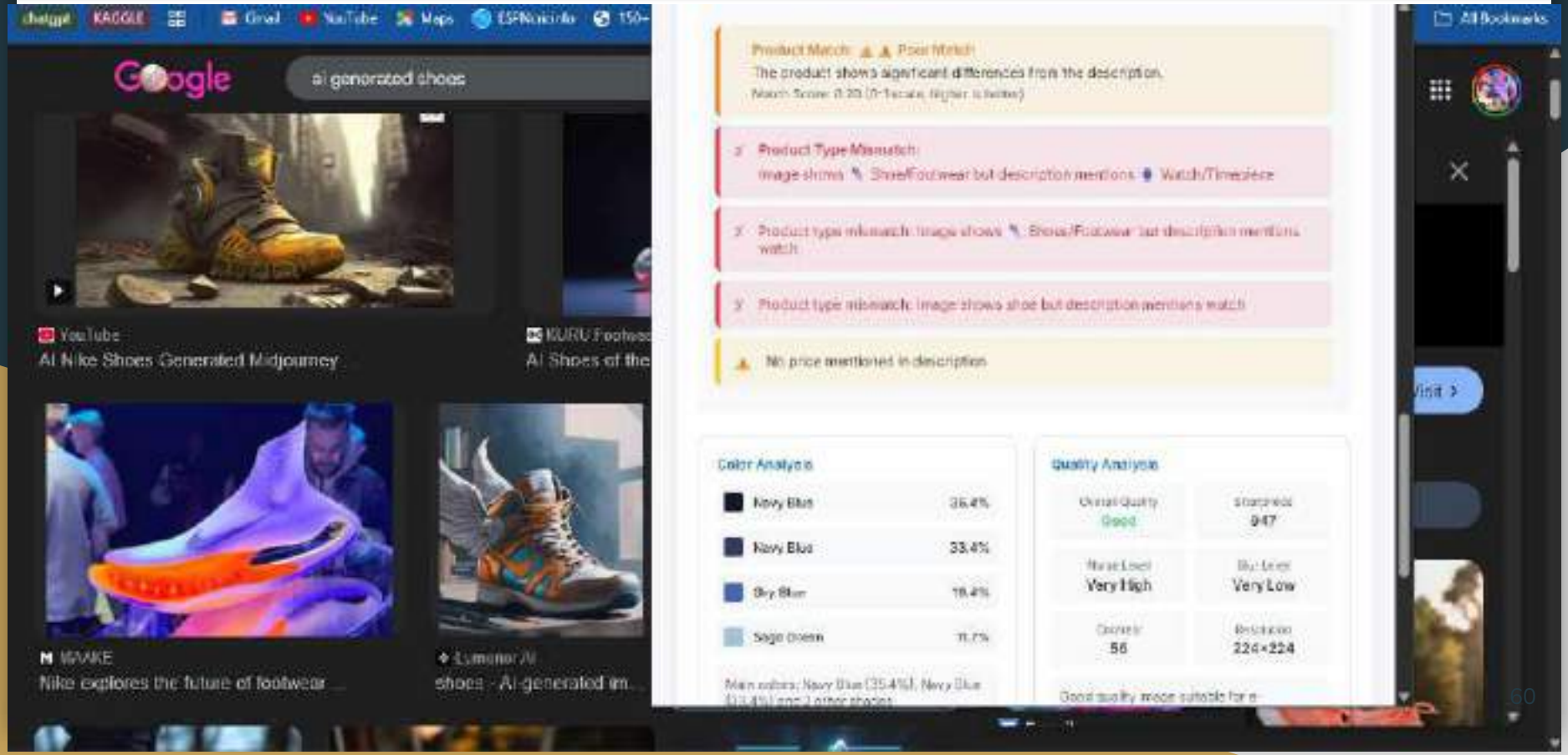
PRODUCT DESCRIPTION ANALYSIS (Real)



PRODUCT DESCRIPTION ANALYSIS (Fake)



PRODUCT DESCRIPTION ANALYSIS (Fake)



ASSUMPTIONS

- **Model Accuracy:** The Deep Learning models (CNNs and Vision Transformers) will effectively detect fake, edited, or AI-generated images.
- **Reliable Explanations:** The Explainable AI tools (Grad-CAM and SHAP) will correctly highlight the suspicious parts of an image.
- **Real-Time Performance:** The system will process and display results quickly, without affecting the user's browsing experience.
- **User Adoption:** Users will install the extension and may contribute by reporting incorrect results.
- **Cross-Platform Support:** The browser extension will work well across various e-commerce websites and browsers.

RISK AND CHALLENGES

- **Evolving Threats:** As generative AI advances, new types of fake images may become harder to detect. Attackers might also attempt adversarial attacks to bypass the system.
- **Data & Accuracy Limitations:** The training dataset may not cover all manipulation types, potentially causing false positives (flagging real images) or false negatives (missing fakes).
- **Performance & Maintenance:** Real-time detection could affect browsing speed, and the system will require regular updates to maintain accuracy and platform compatibility.
- **Privacy & Cost Concerns:** Processing images from users' browsers raises privacy issues, and GPU requirements may increase the cost of large-scale deployment.

CONCLUSION

- ImageGuard combines Deep Learning and Explainable AI (XAI), to effectively detect fake or misleading product images in e-commerce platforms.
- A deep learning model capable of detecting fake and manipulated images in online marketplaces.
- Integration of Explainable AI (XAI) ensured that users could understand and trust the system's decisions.
- Implemented a browser extension for real-time, user-friendly deployment
- The project contributes to enhancing the authenticity and trustworthiness of online marketplaces by providing a reliable tool for fraud detection.

FUTURE PLANS

- Extend the dataset to include additional product categories.
- The system can be extended to support cross-platform deployment, including mobile applications, to increase accessibility for end users.
- Continuous learning mechanisms can be introduced using user feedback to improve detection accuracy over time.
- The model can be adapted to detect emerging AI-generated image techniques by periodic retraining with newly collected data.
- Improve system performance and scalability through further model tuning and larger datasets.

PUBLICATIONS

ImageGuard: An Explainable Hybrid Deep Vision Framework for Counterfeit Image Detection in E-Commerce Marketplaces : Communicated to Ceylon Journal of Science-Scopus Indexed

REFERENCES

[1] AlShariah, N.M., Khader, A. and Saudagar, J., 2019. Detecting fake images on social media using machine learning. International Journal of Advanced Computer Science and Applications, 10(12), pp.170-176.

[2] Ghita, B., Kuzminykh, I., Usama, A., Bakhshi, T. and Marchang, J., 2024, June. Deepfake image detection using vision transformer models. In 2024 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom) (pp. 332-335). IEEE.

[3] Van Zyl, C., Ye, X. and Naidoo, R., 2024. Harnessing eXplainable artificial intelligence for feature selection in time series energy forecasting: A comparative analysis of Grad-CAM and SHAP. Applied Energy, 353, p.122079.

[4] Štancel, M. and Hulič, M., 2019. An introduction to image classification and object detection using YOLO detector. In *CEUR workshop proceedings* (Vol. 2403, pp. 1-8).

[5] Iqbal, J.M., Manikandan, T., Tamilselvan, K.S., Muruganandham, A. and Karthikeyan, S., 2024, July. BERT-Based Deep Learning Models for Enhanced Object Detection in Images. In 2024 Second International Conference on Advances in Information Technology (ICAIT) (Vol. 1, pp. 1-6). IEEE.

THANK YOU